

6

Conditional and Looping Constructs

6.1 INTRODUCTION

In the previous chapters, we have learnt the basics of Python programming where we dealt with simple short programs containing a sequence of instructions given for carrying out a particular task. They were all in a sequence, i.e., one after the other. In real life too, this sequence of operations is observed when a school bus carries schoolchildren from school to home and vice versa (See Fig. 6.1).

There is only one way and the driver has no choice but to follow the road from one bus stop to another and reach home/school by following a sequence.

Similarly, Python also executes one statement after another in a program. All these statements are executed by the CPU. The CPU, as you know, has three parts—ALU (Arithmetic Logic Unit), CU (Control Unit) and MU (Memory Unit).

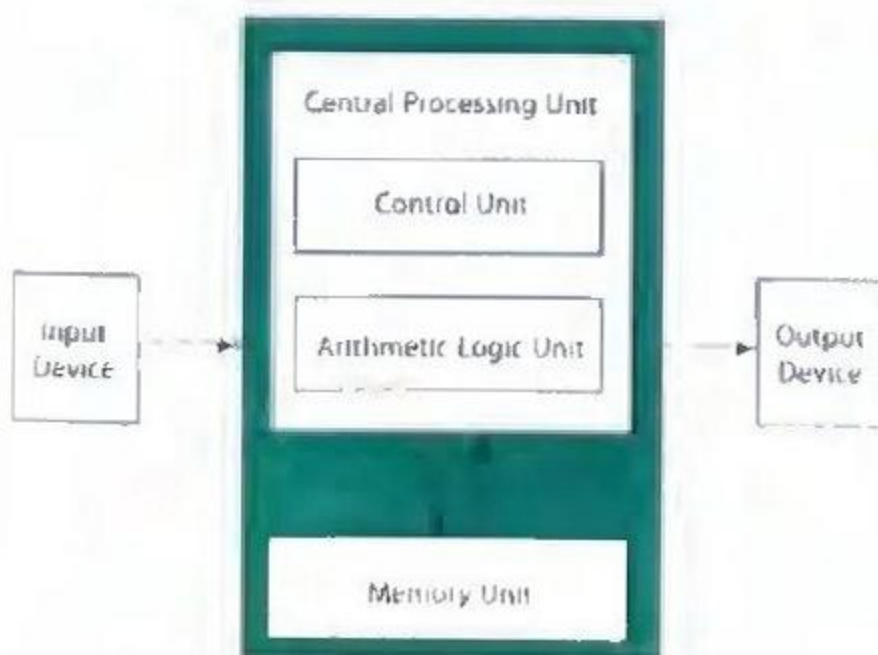


Fig. 6.1: Flow/Sequence in Real World

The Arithmetic Logic Unit involves all mathematical operations and conversions. We will now focus on the Logic Unit which makes a computer think; so we need to write the code.

In real-life scenarios also, we often go with the flow and tend to take decisions, such as after 10th standard, we decide which stream to go for and after completing graduation/post-graduation, one has to decide which company to join. Similarly, if you have ever used Google Maps direction services to reach from

one place to another, you might have noticed that sometimes it shows more than one path, the least crowded path, shortest distance path, etc., and you can decide which path to take. A decision may involve choosing one of the two or more possible options. If it is raining, I will use an umbrella so that I don't get wet (Fig. 6.2). This is a type of branching. If one condition is true, that is, if it is raining, I will take my umbrella. If the condition is not true but false, then I will not take my umbrella.

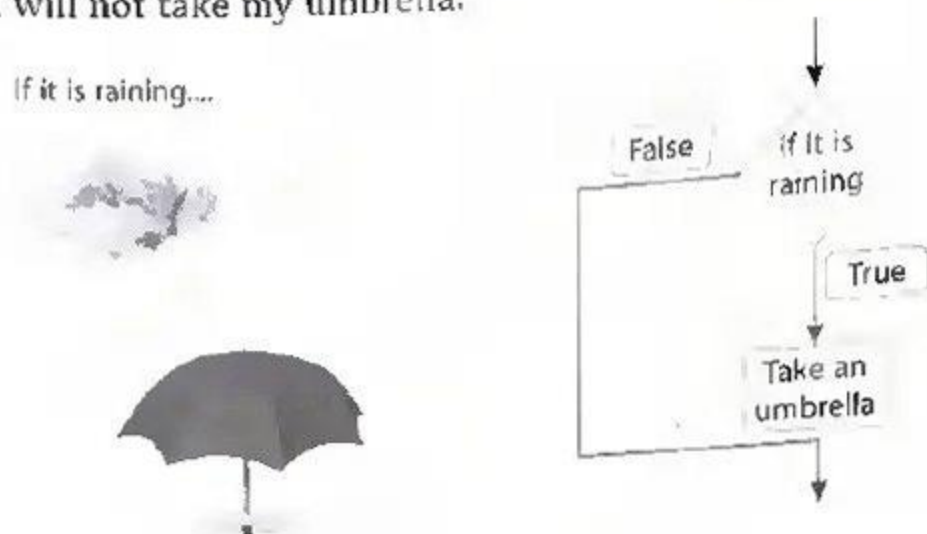


Fig. 6.2: Decision-making

Similarly, in Python also, we have a keyword 'if' which helps in taking decisions. In the above figure, we have used a tool for depicting this decision-making which is a flow chart.

Before we move on to the concept of **program control statements**, which are the Python constructs used to perform repetitive actions and for making decisions, we will discuss the types of statements available in Python.

6.2 TYPES OF STATEMENTS IN PYTHON

Statements are the instructions written in the source code given to the computer for executing any task. These help the user to obtain the desired output by working and manipulating data, making decisions and repeating the set of instructions. Statements constitute the smallest executable unit within a Python program. Python statements are classified into three types:

- ☛ Empty Statement
- ☛ Simple Statement (Single Statement)
- ☛ Compound Statement

1. **Empty Statement:** It is the simplest statement or, more precisely, we can say it is an empty statement which serves as a placeholder in Python. **pass** is a special (empty) statement in Python that does nothing. It only works as a dummy statement.

Its syntax is:

```
pass
```

It can be used when a statement is required syntactically but the program requires **no action** to be performed. Since Python does not support empty code blocks, we can use **pass** statement for the no-operation in the **if-condition** block, **if-else** block and even in **while** loop.

- **Simple Statement:** A simple statement in Python comprises a single logical statement. For example, consider the following single statement:

```
>>> name=input("Enter your Name:")  
>>> print(name)
```

Assignment statements are simple statements like:

- `>>> x = 10`

which means that we are assigning a value "10" to the variable "x". This we call a simple statement.

- The computation statements (*expression statements*) are also called simple statements. These statements will compute or calculate some expressions and return the results.

- For example,

```
>>> x = 10 + 15
```

is an expression statement.

Thus, a simple statement is described as a single executable statement in Python.

- 3. **Compound Statement:** A compound statement is a statement which comprises a group of statements. The compound statements usually execute when a condition is satisfied, or a code block is called directly or through a function call.

Compound statements are spread into multiple logical lines or several single statements but aligned together into a single unit. Its syntax is:

```
<compound statement header>:  
    <multiple simple statements/compound statements  
    indented at same level>
```

Here, header line starts with the keyword and ends at colon (:).

The body consists of more than one simple Python statement or compound statement.

6.3 PROGRAM CONTROL FLOW

The control and flow of a program in any programming language can be broadly classified into three categories:

1. Sequence
2. Selection/Decision
3. Iteration/Looping

In a Python program, statements can be executed sequentially, selectively or iteratively. Now, we shall discuss these program control flow constructs in brief.

1. **Sequential:** Sequential statements in the program are executed in a sequential order or in a linear fashion, one after another, without any jump in the program. In Fig. 6.3, Statement 1 will be executed first, followed by Statement 2 and then by Statement 3. This represents the default flow of statement and is the simplest one.

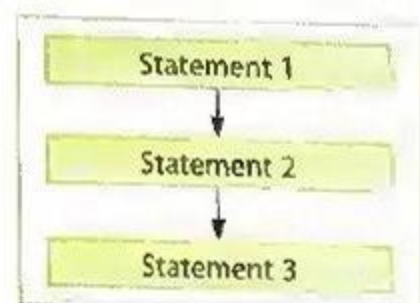


Fig. 6.3: Sequential Statements

2. **Selection Conditional:** Selection conditional statements in a program are executed when the program uses a condition to decide which set of statements is to be executed between two or more alternative sets of statements. In conditional statements 'if..else' statement is used. In Fig 6.4, 'body of if statement' will be executed if test condition evaluates to true otherwise 'body of else' will be executed. These are also called Decision Constructs.

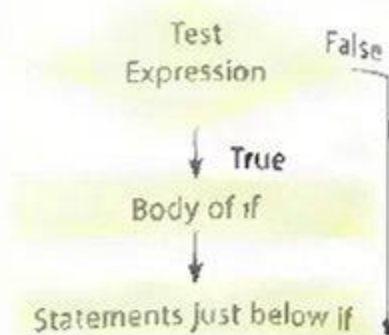


Fig. 6.4: Conditional Statements

3. **Iterative/Looping:** Iterative or Looping statements in a program are executed when a certain set of statements needs to be executed multiple times in a loop. Iterative statements constitute for and while loops. In Fig. 6.5, body of loop will be executed until test expression evaluates to true.

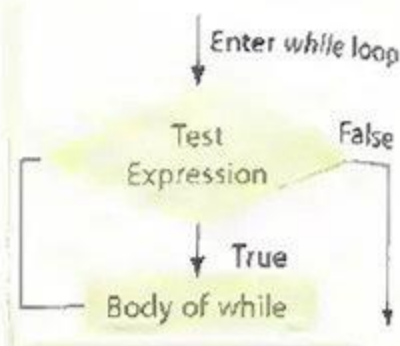


Fig. 6.5: Iterative Statements

The set of statements that is repeated again and again constitutes the body of the loop and this process is termed as iteration. The condition on which the execution or exit of the loop depends is called the **exit** or **test** condition.

6.4 USE OF INDENTATION

In simple terms, indentation refers to adding white space before a statement. Indentation is a very important concept of Python because without proper indenting the Python code, you will end up seeing **IndentationError** and the code will not get compiled.

In most programming languages, the statements within a block are put inside curly brackets. However, Python uses indentation for block as well as for nested block structures. (Fig. 6.6)

Leading whitespace (spaces and tabs) at the beginning of a statement is called indentation. In Python, the same level of indentation associates statements into a single block of code. The interpreter checks indentation levels very strictly and throws up syntax errors if indentation is not correct. It is common practice to use a single tab for each level of indentation.

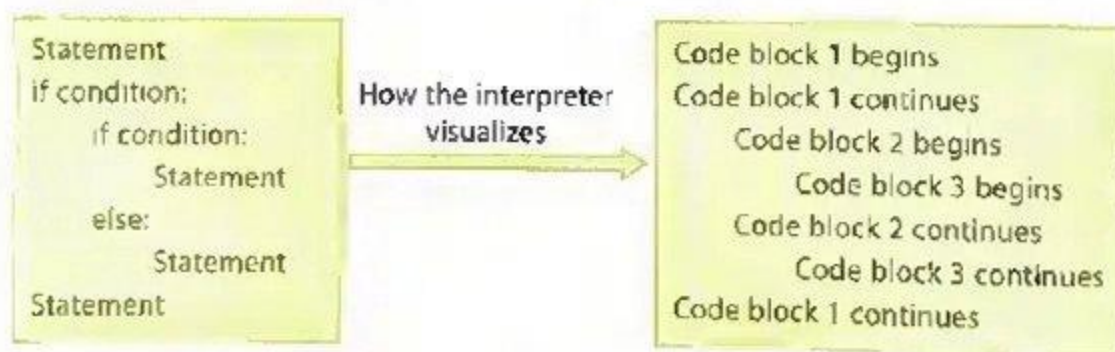


Fig. 6.6: Indentation in Python

In the above example,

- Statement (line 1), if condition (line 2) and statement (last line) belongs to the same block which means that after statement 1, if condition will be executed, and suppose the if condition becomes False, then Python will jump to the last statement for execution.
- The nested if-else belongs to block 2 which means that if nested if becomes False, then Python will execute the statements inside the else condition.

- Statements inside nested if-else belong to block 3 and only one statement will be executed depending on the if-else condition.

Thus, Python indentation is a way of telling a Python interpreter that the group of statements belongs to a particular block of code.

6.5 CONDITIONAL STATEMENTS

The order of execution of the statements in a program is known as **flow of control**. It is dependent on the programming constructs used while writing the program. Decision-making or conditional statements are used to control the flow of execution of a program depending upon the condition.

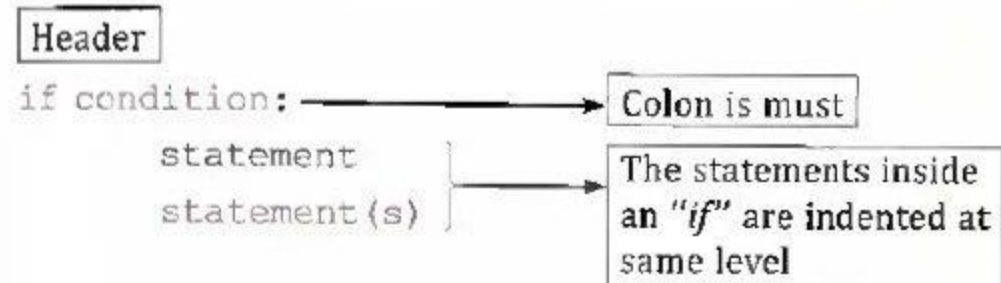
➤ There are four types of conditional statements in Python:

1. *if* statement
2. *if-else* statement
3. *if-elif-else* statement
4. Nested *if-else* statement

6.5.1 if Statement

if statement is the easiest method for making a decision in Python. It simply states that if something is true, Python should perform the steps that follow and do nothing in case the condition evaluates to false. In programming, decision-making or selection can be achieved through the conditional statement. The simplest form is the *if* statement.

Syntax:



- If the condition is true, then the indented statement gets executed.
- The indented statement implies that its execution is dependent on the header.
- There is no limit on the number of statements that can appear under an *if* block.
 - '*if*' is a keyword, followed by a condition and ended with 'colon'.
 - We can use the condition with bracket () also but it is not mandatory.
 - A statement or statements (Block of code) are indented (usually 4 spaces) inside the '*if*' statement and are executed only if the condition evaluates to true, otherwise the control skips the block of statements inside *if* statement and only statement2 will be executed.

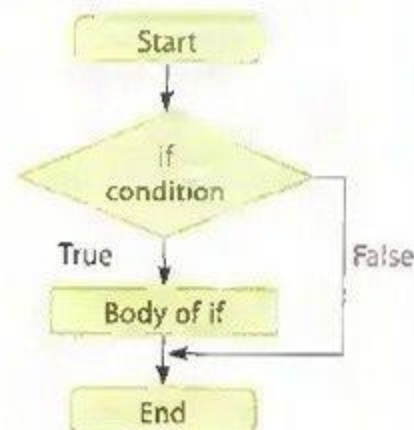
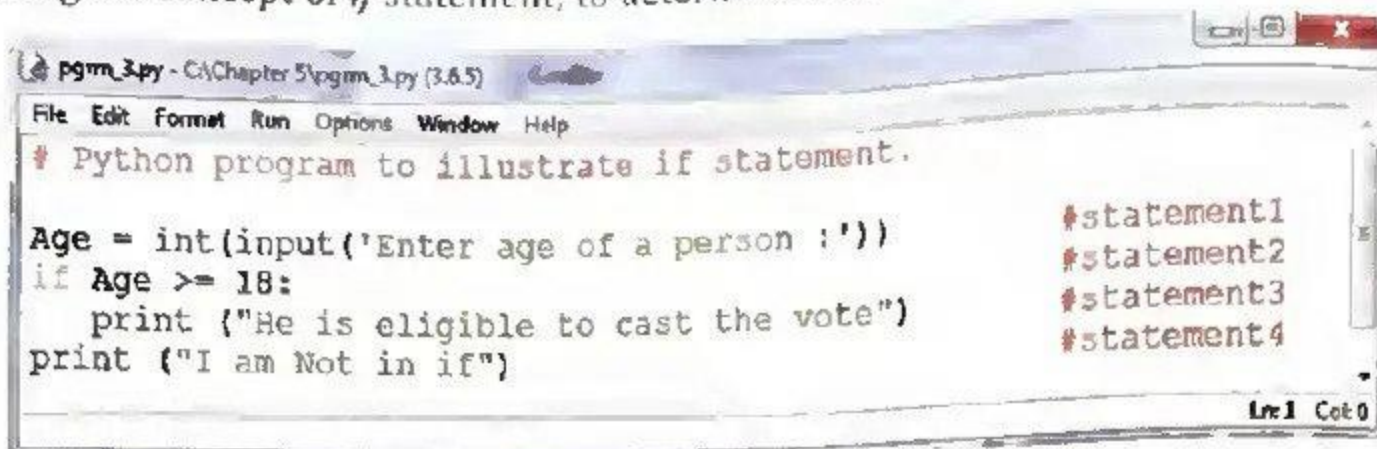


Fig. 6.7(a): Flow Chart for if Statement

Let us look at an example to understand if statement in Python.

Practical Implementation-1

Illustrating the concept of *if* statement, to determine whether a person is eligible to vote.



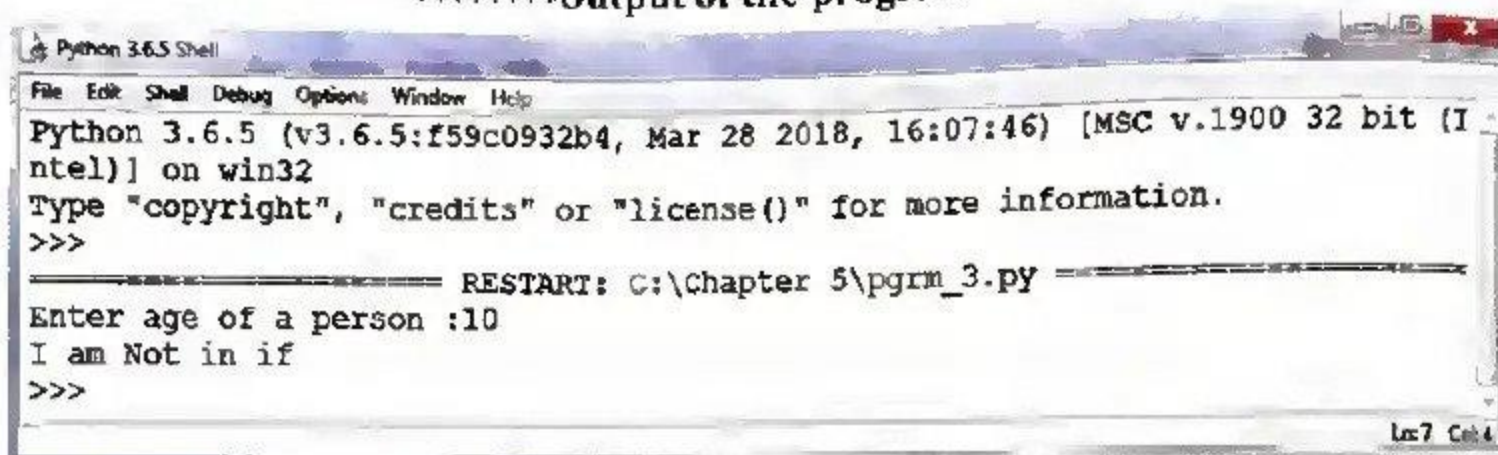
```
pgm_3.py - C:\Chapter 5\pgm_3.py (3.6.5)
File Edit Format Run Options Window Help
# Python program to illustrate if statement.

Age = int(input('Enter age of a person :'))
if Age >= 18:
    print ("He is eligible to cast the vote")
print ("I am Not in if")

#statement1
#statement2
#statement3
#statement4

Ln:1 Col:0
```

*****Output of the program*****



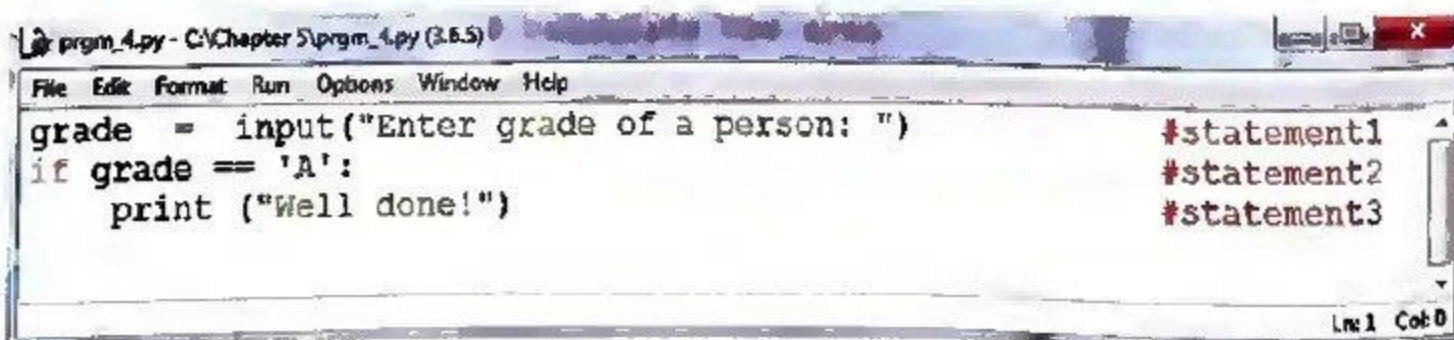
```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Chapter 5\pgm_3.py =====
Enter age of a person :10
I am Not in if
>>>
```

Explanation:

1. statement1 asks the user to input age.
2. statement2 Age >= 18 is the test expression.
3. The body of if, i.e., statement3 is executed only when if condition evaluates to true.
4. When variable 'Age' is equal to 10, i.e., the inputted age is less than 18, the test expression becomes false and body of if statement is skipped, i.e., statement3 does not get executed at all.
5. statement4 falls outside of the if block (un-indented). Hence, it is executed regardless of the test expression.

Practical Implementation-2

Illustrating the concept of *if* statement, display the grade of the topper in the class.



```
pgm_4.py - C:\Chapter 5\pgm_4.py (3.6.5)
File Edit Format Run Options Window Help
grade = input("Enter grade of a person: ")
if grade == 'A':
    print ("Well done!")

#statement1
#statement2
#statement3

Ln:1 Col:0
```

*****Output of the program*****

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46)
[MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more info
rmation.
>>>
===== RESTART: C:\Chapter 5\prgm_4.py =
=====
Enter grade of a person: A
Well done!
>>>

```

Now consider a situation when the user inputs grade other than 'A', then nothing shall be executed, and the flow control of the program shall terminate on the condition evaluated as false without notifying anything to the user.

Here, we need a second type of conditional *if* statement called *if...else* that provides an alternative execution. *if...else* allows to have two possibilities and the condition determines which branch gets executed.

Learning Tip: A statement or statements (Block of code) indented (usually 4 spaces) inside the *if* statement are executed only *if* condition evaluates to true, otherwise the control skips the block of statements inside *if* and control goes to the outside of the *if* block (un-indented).

Therefore, we will improvise the above programs for the false situations by introducing the '*else*' part in association with the *if* statement.

6.5.2 if-else Statement

'*if*' statement alone tells us that if a condition is true, it will execute a block of statements and if the condition is false, it won't. But what if we want the program to display some appropriate message or to execute certain set of instructions in case the condition becomes false. Here comes the *else* statement. We can use the *else* statement with *if* statement to execute a block of code when the condition is false as shown in Fig. 6.7(b).

Syntax:

```

if condition:
    statement(s)
else:
    statement(s)

```

Here:

- '*if*' and '*else*' are keywords.
- A statement or statements (Block of code) indented (usually 4 spaces) inside the *if* statement are executed only if condition evaluates to true, otherwise statements following *else* will be executed.

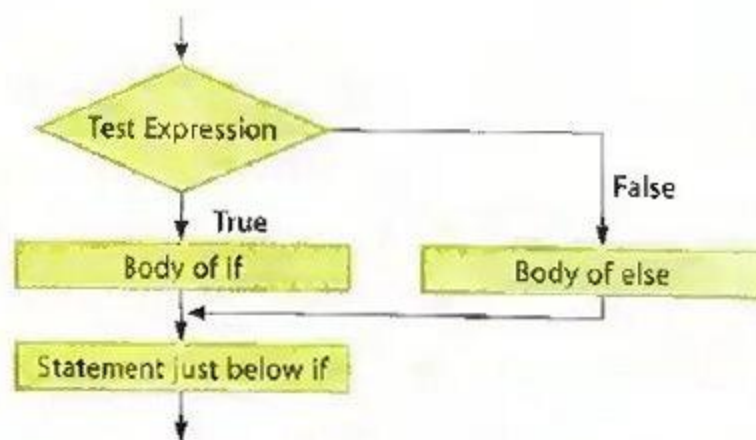
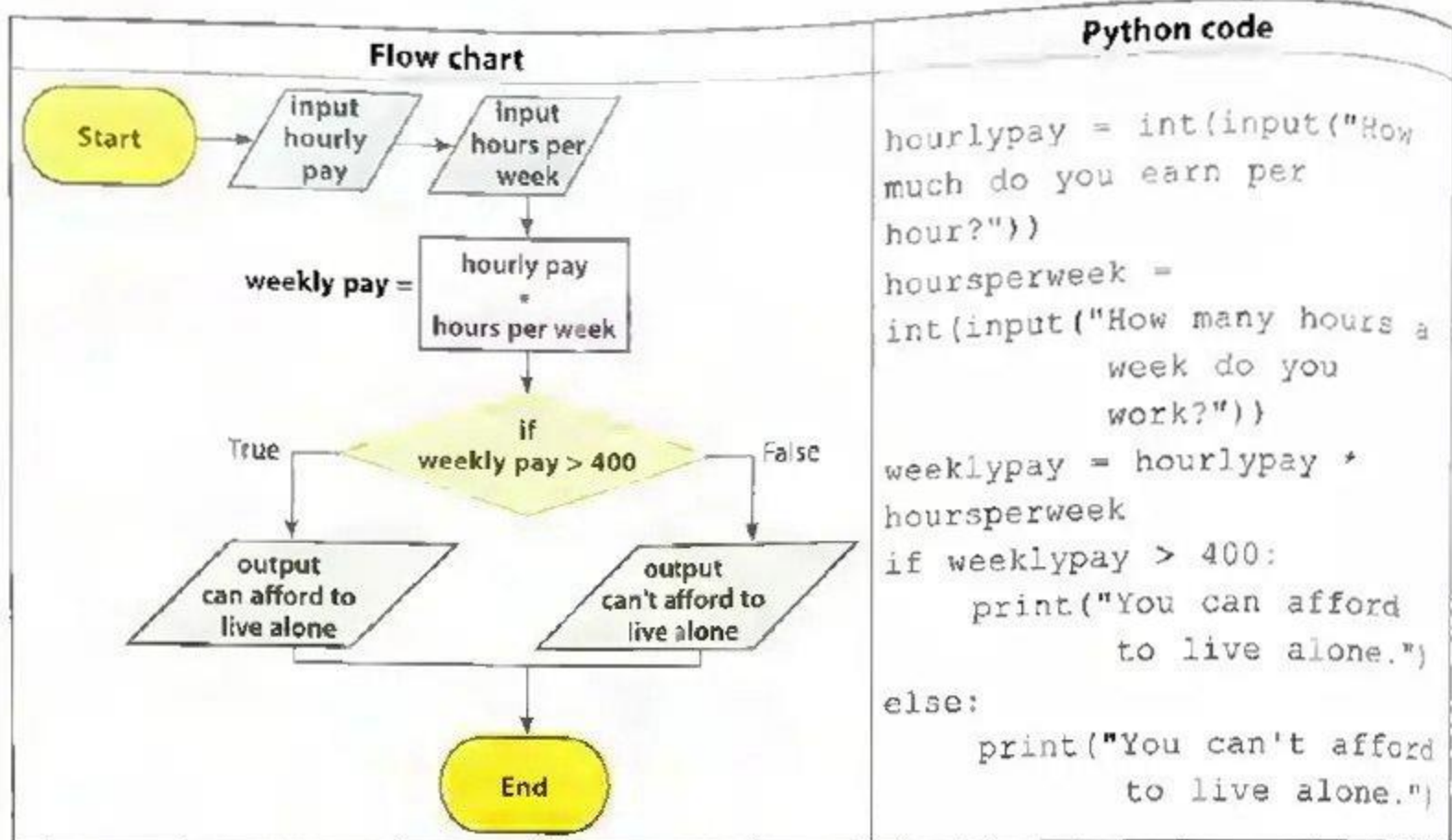


Fig. 6.7(b): Flow Chart for if-else Statement

Let us consider an example that explains if-else construct along with its associated pseudo-code and flow chart.

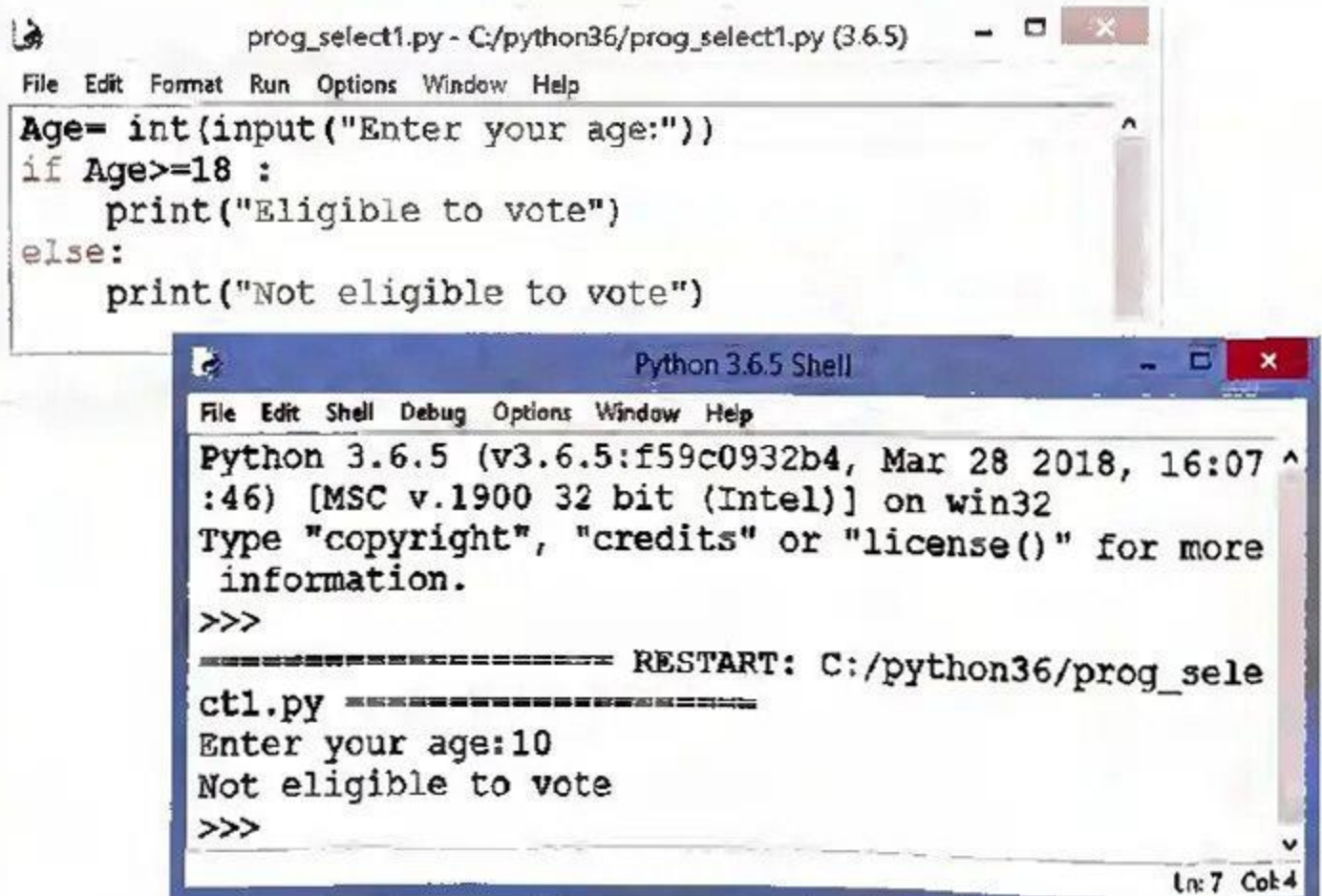
Example 1:

To develop a program that asks the user to input the number of working hours and earnings per day. It then calculates how much money they earn in a week and display an appropriate message based on their earnings.



Practical Implementation-3

Illustrating the concept of *if-else* statement. (Modification of Practical Implementation-1).



```
prog_select1.py - C:/python36/prog_select1.py (3.6.5)
File Edit Format Run Options Window Help
Age= int(input("Enter your age:"))
if Age>=18 :
    print("Eligible to vote")
else:
    print("Not eligible to vote")

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07 ^
:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more
information.
>>>
===== RESTART: C:/python36/prog_sele
ct1.py =====
Enter your age:10
Not eligible to vote
>>>
```

Let's take another example to gain a better understanding of *if-else* statement.

Practical Implementation-4

Write a code to find whether a given number is odd or even.

```
pgm_6.py - C:\chapter 5\pgm_6.py (3/5)
File Edit Format Run Options Window Help
x = int(input("Enter no:"))
if (x%2==0):
    print ("Even number")
else:
    print ("Odd number")
Ln 1 Col 6
```

*****Output of the program*****

```
Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:\chapter 5\pgm_6.py
Enter no:20
Even number
>>>
Ln 7 Col 4
```

Many a time, there are situations that require multiple conditions to be checked and hence may lead to many alternatives. In such a case, we can chain the conditions using *if...elif*.

Practical Implementation-5

Program to find absolute value of a number in Python.

```
prog_absolute_value.py - C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\pro...
File Edit Format Run Options Window Help
#Program to find absolute value of a number in Python
x = int(input("Enter the number: "))
if x > 0:
    print("absolute value is",x)
else:
    print("absolute value is",-x)
Ln 16 Col 4
```

```
>>>
RESTART: C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\pro...
e_value.py
Enter the number: -50
absolute value is 50
```

Practical Implementation-6

Program to check divisibility of a number.

```
prog_divi-copy - C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\pro...
File Edit Format Run Options Window Help
#Program to check divisibility of a number
print("Enter a Number (Numerator): ")
numn = int(input())
print("Enter a Number (denominator): ")
numd = int(input())

if numn%numd==0:
    print("\n" +str(numn)+ " is divisible by " +str(numd))
else:
    print("\n" +str(numn)+ " is not divisible by " +str(numd))
Ln 18 Col 2
```

```
>>>
RESTART: C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\pro...
.py
Enter a Number (Numerator):
20
Enter a Number (denominator):
13
20 is not divisible by 13
```

```
Enter a Number (Numerator):
40
Enter a Number (denominator):
8
40 is divisible by 8
```

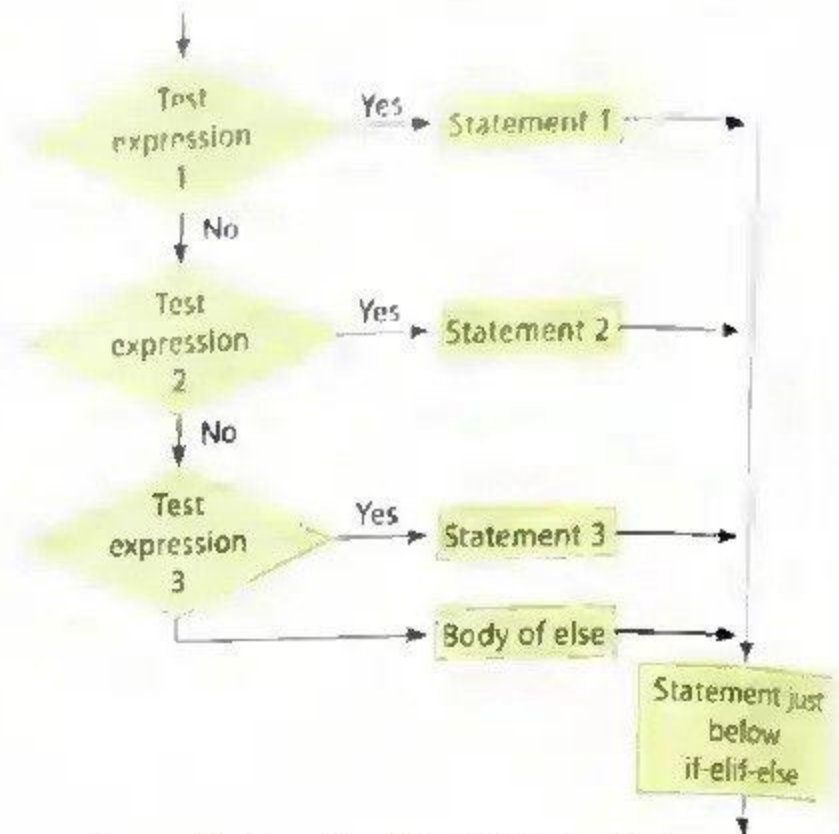
6.5.3 if-elif-else Statement

if-elif-else construct is used in the situations where a user can decide among multiple options. As soon as one of the conditions controlling the *if* is true, then statement/statements associated with that *if* are executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final *else* statement will be executed, and control flows from top to down.

Learning Tip: *if-elif-else* statement—Each condition is checked in order. If the first is false, the next is checked and so on. If one of them is true, the corresponding branch executes, and the statement ends. Even if more than one condition is true, only the first true branch executes.

Syntax:

```
if condition:  
    statement(s)  
elif condition:  
    statement(s)  
elif condition:  
    statement(s)  
else:  
    statement(s)
```



Here:

- *else* is optional
- Number of *elif* is dependent on the number of conditions
- If the first condition is false, the next is checked, and so on. If one of the conditions is true, the corresponding statement(s) executes, and the statement ends.

Fig. 6.7(c): Flow Chart for if-elif-else Statement

Practical Implementation-7

Illustrating the concept of *if-elif-else* statement, accept the percentage of a student and display the grade accordingly.

```
prgm_T.py - Chapter 5 prgm_T.py (3.6.3)
File Edit Format Run Options Window Help
# A program to accept percentage of a student and display grade
perc = float(input("enter percentage of a student : ")) # statement1
if perc > 85: # condition1
    print ('A')
elif perc > 70 and perc <=85: # condition2
    print ('B')
elif perc > 60 and perc <=70: # condition3
    print ('C')
elif perc > 45 and perc <=60: # condition4
    print ('D')
else:
    print ('E')
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Chapter 5\prgm_7.py =====
enter percentage of a student : 78
8
>>>
```

Explanation:

1. statement1 accepts the percentage as the input from the user.
2. Each condition is checked in order. As per the inputted value of 78 as the percentage, condition1 is false but condition2 is true and, hence, the corresponding code of condition2 will be executed and control will not even check other conditions.

Another example explaining the *if-elif-else* construct is as follows:

Practical Implementation-8

Illustrating the concept of *if-elif-else* statement, accept the salary as input from the user and calculate the tax payable.

```
prgm_8.py - C:\Chapter 5\prgm_8.py (3.6.5)
File Edit Format Run Options Window Help
# program to accept salary and calculate tax amount
salary= int(input("Enter salary of a person : "))          # statement1
if salary <=50000:                                         # condition1
    tax=0.05*salary
elif salary<=60000:                                       # condition2
    tax=0.07*salary
elif salary<=70000:                                       # condition3
    tax=0.08*salary
else:
    tax=0.10*salary
print ("Salary : ",salary," Tax : ",tax)
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Chapter 5\prgm_8.py =====
Enter salary of a person : 65000
Salary : 65000 Tax : 5200.0
>>>
```

Practical Implementation-9

Program to sort three inputted numbers and arrange them in ascending order.

```
prog_sort_3nos.py - C:/Users/preeti/AppData/Local/Programs/Python/Python...
File Edit Format Run Options Window Help
#Program to sort three inputted numbers and arrange
#them in ascending order.
x = int(input("Enter first number: "))
y = int(input("Enter second number: "))
z = int(input("Enter third number: "))
if y>=x<=z:
    if y<=z:
        min,mid,max=x,y,z
    else:
        min,mid,max=x,z,y
elif x>=y<=z:
    if x<=z:
        min,mid,max=y,x,z
    else:
        min,mid,max=y,z,x
elif x>=z<=y:
    if x<=y:
        min,mid,max=z,x,y
    else:
        min,mid,max=z,y,x
print("Numbers in ascending order = ",min,mid,max)

>>>
RESTART: C:/Users/preeti/AppData/Local
os.py
Enter first number: 67
Enter second number: 5
Enter third number: 14
Numbers in ascending order = 5 14 67
```

6.5.4 Nested if-elif-else Statement

The nested *if...elif...else* statement allows you to check for multiple test expressions and execute different codes for more than two conditions. We can have many levels of nesting inside *if...else* statements.

Syntax	Example
<pre>if condition: statement(s) if condition: statement(s) elif condition: statement(s) else: statement(s) elif condition: statement(s) else: statement(s)</pre>	<pre>var = 100 if var < 200: print("value is less than 200") if var == 150: print("Which is 150") elif var == 100: print("Which is 100") elif var == 50: print("Which is 50") elif var < 50: print("value is less than 50") else: print("Could not find true countenance") print("Good bye!")</pre>

We can have an *if...elif...else* statement inside another *if...elif...else* statement. This is called nesting in computer programming.

Any number of these statements can be nested inside one another. Indentation is the only way to figure out the level of nesting. This can get confusing, so it must be avoided as far as possible.

Practical Implementation-10

Illustrating the concept of nested *if-else* statement, find whether an inputted number is a 0 (zero), a positive number or a negative number.

```
*prgm_9.py - C:\Chapter 5\prgm_9.py (3.6.5)
File Edit Format Run Options Window Help
num = float(input("Enter a number: ")) #statement1
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")
Ln 1 Col 30
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 3
2 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:\Chapter 5\prgm_9.py
Enter a number: 5
Positive number
>>>
Ln 7 Col 4
```

Explanation:

1. statement1 asks the user to input a number.
2. Check if the number is positive or negative or zero and display an appropriate message.

Till now, we have learnt how to make a decision. To take one path or another is an essential element of performing useful work. The programs we have worked upon till now have performed a series of steps just one time and then stopped. However, the real world doesn't work this way. Many of the tasks that humans perform are repetitious. *For example*, the doctor might state that you need to exercise more and tell you to do 100 push-ups each day. If you just do one push-up, you won't get much benefit from the exercise and you definitely won't be following the doctor's order. Of course, because you know precisely how many push-ups to do, you can perform the task a specific number of times. Python allows the same sort of repetition using Iteration/Looping statements which we will discuss now.

6.6 ITERATION

In our day-to-day life, we repeat tasks. *For example*, we repeat paying electricity bill every month or, as shown in Fig. 6.8, a silkworm lays eggs, turns into a caterpillar, becomes a pupa, matures as a silk moth and lays eggs again and the cycle starts again.

This kind of **repetition** is also called **iteration**. Repetition of a set of statements in a program is made possible using looping constructs.

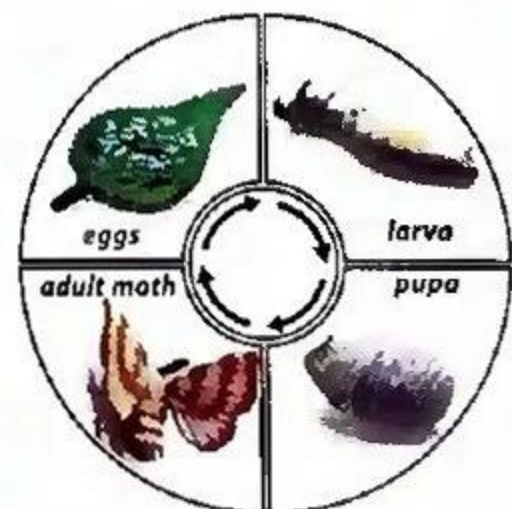


Fig. 6.8: Life Cycle of a Silkworm (Iterative Process)

To understand further, let us look at the following Python program that prints the first five natural numbers:

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

Output:

```
1
2
3
4
5
```

What would you do if you were asked to print the first 1,00,000 natural numbers? Writing 1,00,000 print statements would not be an efficient solution. It would be tedious and not the right way to do. Writing a program having a loop is a better solution here. The pseudo-code is given below:

1. Take a variable *count* and set its value to 1.
2. Print the value of *count*.
3. Check the condition ($\text{count} \leq 100000$)
4. Increment the variable ($\text{count} += 1$).
5. Repeat the steps 2 to 4 as long as *count* has a value equal to 1,00,000.

Looping constructs provide the facility to execute a set of statements in a program repetitively, based on a condition. The statements in a loop are executed again and again as long as a particular logical condition remains true. This condition is checked through a variable called control variable. When the condition becomes false, the loop terminates. It is the responsibility of the programmer to ensure that this condition eventually does become false, i.e., there is an exit condition and the loop does not continue endlessly. For example, if we didn't set the condition $\text{count} \leq 1,00,000$, the program would have never stopped.

- A loop statement allows us to execute a statement or a group of statements multiple times. The adjacent diagram (Fig. 6.9) illustrates a loop statement.

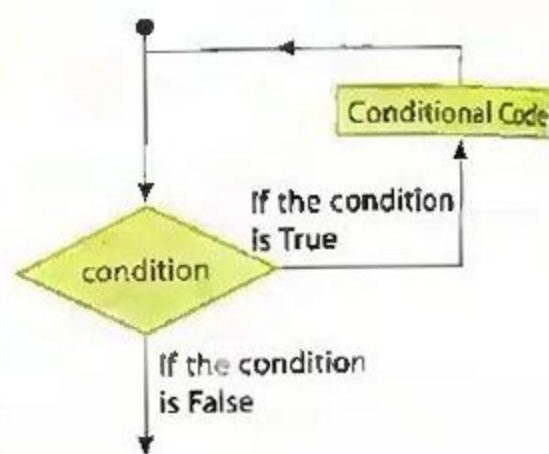


Fig. 6.9: Flow Chart for a Loop

Types of loops

Python programming language provides the following types of loops to handle looping requirements.

1. *for* loop
2. *while* loop

'while' loops are also called 'indefinite loops' because statements are executed until a logical condition remains true. 'for' loops are called 'definite loops' because statements are executed for a definite number of times.

6.6.1 for Loop

The *for* loop statement is used to iterate/repeat itself over a range of values or a sequence. The *for* loop is executed for each of these items in the range. These values can be either numeric or, as we shall see in successive chapters, elements of a data structure like a string, list, or tuple.

With every iteration of the loop, the control variable checks whether each of the values in the range has been traversed or not. When all the items in the range are exhausted, the body of the loop is not executed anymore; the control is then transferred to the statement immediately following the *for* loop.

In *for* loop, it is known before hand how many times the loop is going to be executed as depicted in the flow chart given in Fig. 6.10.

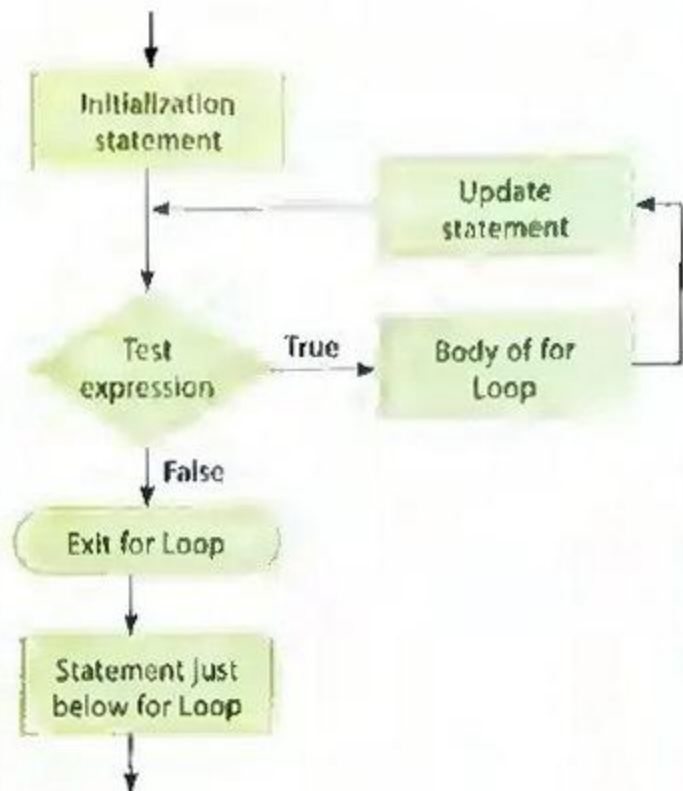


Fig. 6.10: Flow Chart Depicting a *for* Loop

Syntax of *for* Loop:

```
for <Control_variable> in <sequence/items in range>:
    <statements in body of loop>
else:          # optional
    <statements>
```

Here:

- sequence may be a list, string, tuple.
- 'else' statement will be executed after all the iterations of *for* loop, if provided.
- Control_variable is a variable that takes a new value from the range each time loop is executed.

Example 1: for letter in 'PYTHON': print(letter)	Output: P Y T H O N
Example 2: count = [10,20,30,40,50] for c in count: print(c)	Output: 10 20 30 40 50
Example 3: numbers=[1,2,3,4,5,6,7,8,9,10] for n in numbers: if n % 2 == 0: print(n, 'is an Even Number')	Output: 2 is an Even Number 4 is an Even Number 6 is an Even Number 8 is an Even Number 10 is an Even Number

Note that the body of the loop is indented with respect to the *for* statement.

Practical Implementation-11

Illustrating the concept of *for* loop statement.

```
pgm_13.py - C:/Chapter 5/pgm_13.py (3.6.5)
File Edit Format Run Options Window Help
# Python program to illustrate for loop statement
for i in '123':
    print ("Welcome",i,"times") # body of for loop
Ln:1 Col:0
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Chapter 5/pgm_13.py
Welcome 1 times
Welcome 2 times
Welcome 3 times
>>>
Ln:8 Col:4
```

Explanation:

1. Here *i* is loop control variable and will be assigned each value of a string '123', i.e., for the first time *i* will be 1, then 2 and then 3.
2. Body of *for* loop will be executed for each value of loop variable *i*, i.e., 3 times.
3. In the first iteration, the loop shall get executed with *i*=1 and will print Welcome 1 times.
4. Then body of the loop will be executed with *i*=2 and will print Welcome 2 times.
5. Again the same method of execution shall take place with *i*=3 and will print Welcome 3 times.

Practical Implementation-12

Illustrating the concept of *for* loop statement.

```
prog_select2.py - C:/python36/prog_select2.py (3.6.5)
File Edit Format Run Options Window Help
# Python program to demonstrate 'for' loop
for ch in 'GOA':
    print(ch)
else:
    print('Coming out of the loop')
```

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/python36/prog_select2.py
G
O
A
Coming out of the loop
>>>
Ln:9 Col:4
```

Explanation:

In the above program, *for* loop is used to traverse through the string 'GOA' character and gets printed on separate line. Once the *for* loop iterations get over, the *else* part gets executed and finally the message "Coming out of the loop" gets printed.

1. The range function used in the above program will generate values as 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.
2. 'i' is loop control variable and will be assigned each generated value, for which the 'body of *for* loop' will be executed.

Practical Implementation-13

Program to generate the table of a number using *for* loop.

```
pgm_15.py - C:\Chapter 5\pgm_15.py (3.6.5)
File Edit Format Run Options Window Help
# Python program to print table of a number, say 3
num=3
for i in range(1,11):
    print (num,' x ',i,' = ', num*i)
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:\Chapter 5\pgm_15.py
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
>>>
```

Explanation:

1. The range function used in the above program will generate values as 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.
2. 'i' is loop control variable and will be assigned each generated value, for which the 'body of *for* loop' will be executed.

Practical Implementation-14

To calculate and display the factorial of an inputted number.

```
pgm_16.py - C:\Chapter 5\pgm_16.py (3.6.5)
File Edit Format Run Options Window Help
# Python program to accept a number from a user and print factorial of that number.
num=int(input("Enter non negative no to take factorial of : "))
fact = 1
for i in range(1,num):
    fact = fact * (i+1)
print ('Factorial of ',num,' = ',fact)
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b6, Mar 28 2018, 16:07:14) [MSC v.1900 32 bit (Int
el)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
----- RESTART: C:/Chapter 5/prgm_16.py -----
Enter non negative no to take factorial of : 8
Factorial of 8 = 40320
>>>
```

POINTS TO REMEMBER

- The **for** loop in Python is used to iterate values from range of values, strings, lists, tuples or dictionary.
- 'else' statement is optional and will be executed after all the iterations of **for** loop are over.
- Repeated execution of a set of statements is called iteration.
- The loop variable contains the highest value of the list after '**for**' loop is over.
- Value of **for** loop variable cannot be defined beforehand.

You must have observed that in the above examples we have used the **range()** function which generates the values as 1, 2, 3, 4,... and so on till the limit defined in the loop statement. We shall now discuss **range()** function in detail.

range() Function

The **range()** is a built-in function in Python and is used to create a list containing a sequence of numbers starting with the **start** and ending with one less than the **stop**. We have already used this function in the previous Practical Implementation.

Syntax:

```
range([start], stop[, step])
```

The **start** and **step** parameters are optional. By default, the list starts from 0 and in every iteration, it is incremented by one but we can specify a different increment value by using the **step** parameter. **range()** is often used in **for** loops.

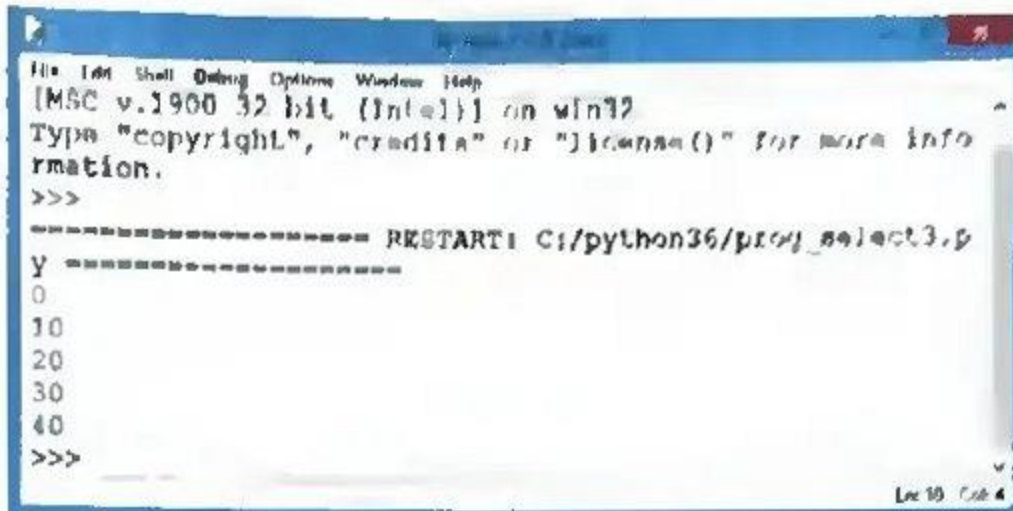
For example,

Command	Values generated
>>> range(10)	[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> range(1, 11)	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> range(0, 30, 5)	[0, 5, 10, 15, 20, 25]
>>> range(0, -9, -1)	[0, -1, -2, -3, -4, -5, -6, -7, -8]
>>> list(range(2, 10))	[2, 3, 4, 5, 6, 7, 8, 9] #start value is given as 2
>>> list(range(0, 30, 5))	[0, 5, 10, 15, 20, 25] #step value is 5 and start value is 0
>>> list(range(0, -9, -1))	[0, -1, -2, -3, -4, -5, -6, -7, -8] #step value is -1. Hence, decreasing #sequence is generated

The arguments of **range()** function must be integers. The **step** parameter can be any positive or negative integer other than zero.

for example,

```
prog_select.py C:/python36/program select.py (364) - n
File Edit Format Run Options Window Help
# Program to demonstrate range() function
for n in range(5):
    print(n*10)
```



```
File Edit Shell Debug Options Window Help
[MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "help()" for more info
>>>
----- RESTART: C:/python36/program select.py -----
>>>
0
10
20
30
40
>>>
```

6.6.2 while Loop

A *while* statement in Python executes a block of code repeatedly as long as the test/control condition of the loop is true. The control condition of the *while* loop is executed before any statement in the body of the loop is executed. If the condition is true, the body of the loop is executed. Again the control condition of the loop is tested and the loop continues as long as the condition remains true. When the test outcome of this condition becomes false, the loop is not entered again and the control is transferred to the statement immediately following the body of the loop as shown in the flow chart given in Fig. 6.11.

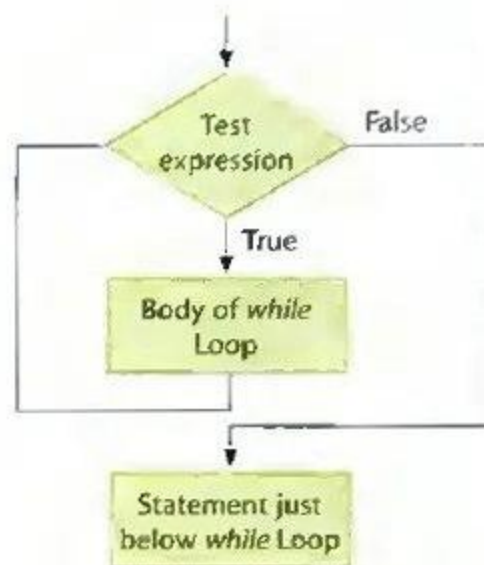


Fig. 6.11: Flow Chart Depicting a *while* Loop

Syntax of *while* loop:

```
while <test_expression>:
    Body of while
else:      # optional
    Body of else
```

Here:

- '*while*' is a reserved word.
- *test_expression* is the condition to be checked for true/false value.
- '*Body of while*' consists of single or multiple statements.
- '*else*' is optional and executed when test condition evaluates to false.

Example 1:

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

Output:

```
1
2
3
4
5
```

Example 2:

```
n = int(input(" Enter a number to find
its factor : "))
print(1, end=' ')
# one is a factor of every number
factor = 2
while factor <= n/2 :
    if n % factor == 0:
        print(factor, end=' ')
        factor += 1
print(n, end=' ')
# every number is a factor of itself
```

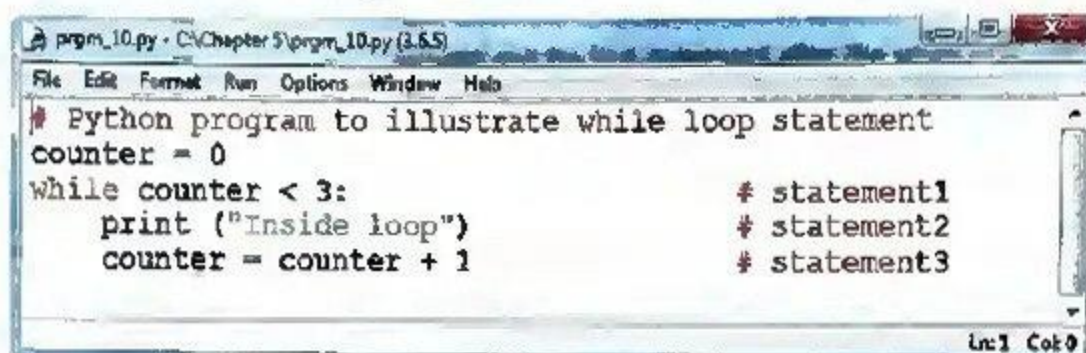
Output.

```
Enter a number to find its factor :
1 2 3 6
```

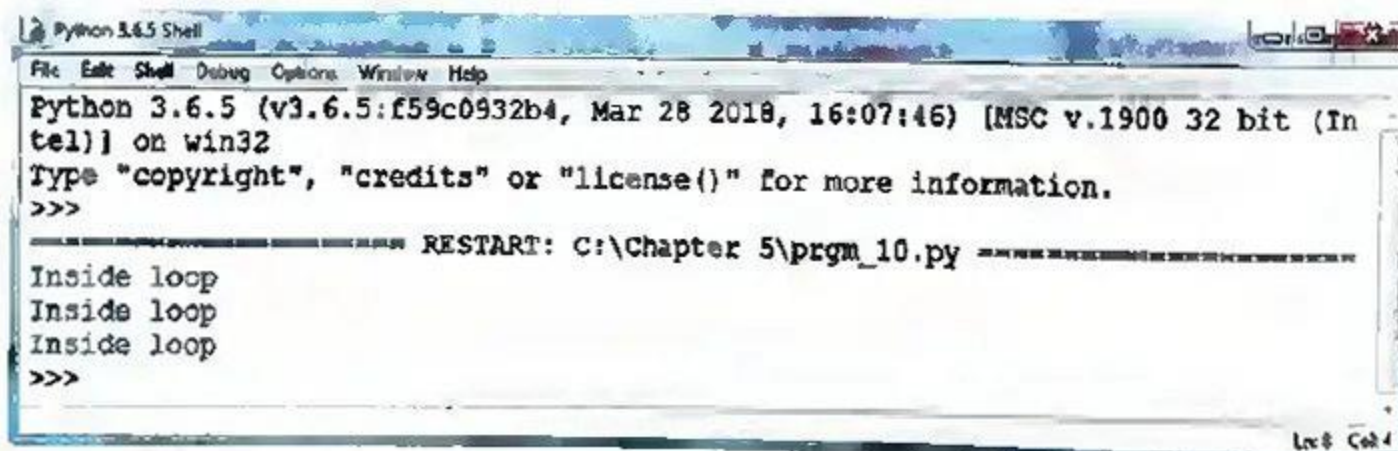
Note that body of the loop is indented with respect to the *while* statement.

Practical Implementation-15

Illustrating the concept of *while* loop statement.



```
prgm_10.py - C:\Chapter 5\prgm_10.py (3.6.5)
File Edit Format Run Options Window Help
# Python program to illustrate while loop statement
counter = 0
while counter < 3:
    print ("Inside loop")
    counter = counter + 1
# statement1
# statement2
# statement3
Ln:1 Col:0
```

*******Output of the program*******


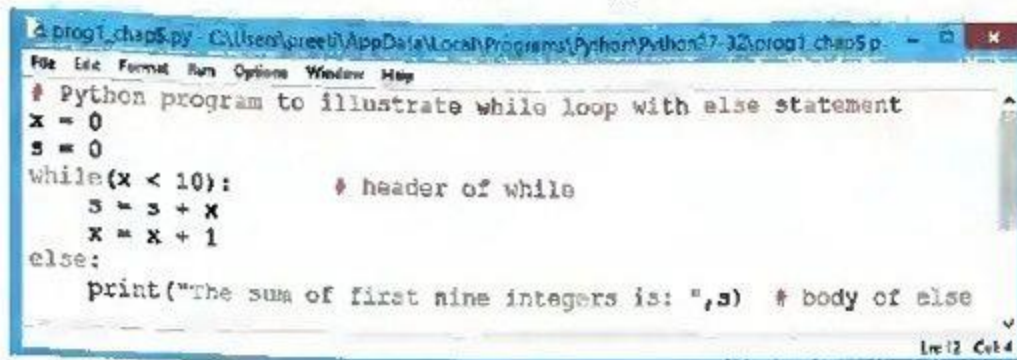
```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (In
tel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:\Chapter 5\prgm_10.py
Inside loop
Inside loop
Inside loop
>>>
Ln:8 Col:4
```

In the above program, loop control variable 'counter' is initialized with 0. It checks for the value of counter up to 3, the loop shall be executing till this value. Every time the loop gets executed, the message "Inside loop" gets displayed, followed by incrementing the value of counter in each iteration. Thus, this loop shall iterate for three times and shall terminate as the value of counter becomes 3.

Till now, we have discussed the *while* loop construct without the use of else part. Now we will see how else construct performs well with the *while* loop.

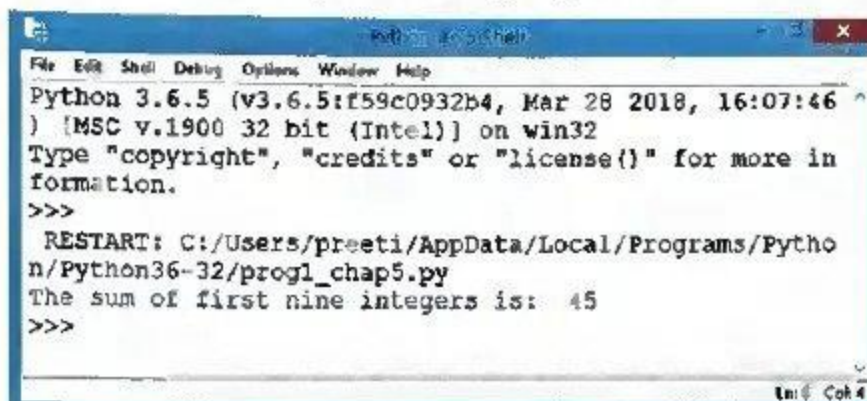
Practical Implementation-16

Illustrating the concept of *while* loop statement using *else*.



```
prog1_chap5.py C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\prog1_chap5.py
File Edit Format Run Options Window Help
# Python program to illustrate while loop with else statement
x = 0
s = 0
while(x < 10):          # header of while
    s = s + x
    x = x + 1
else:
    print("The sum of first nine integers is: ",s) # body of else
Ln 12 Col 4
```

*****Output of the program*****



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46)
[MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more in
formation.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/prog1_chap5.py
The sum of first nine integers is: 45
>>>
```

Note that control will go to '*else*' part only when condition of '*while*' statement becomes false.

POINTS TO REMEMBER

- The loop control variable must be updated inside the loop to avoid endless (infinite) loop, which we will discuss now.
- In Python, the body of the *while* loop is determined through indentation.
- Body starts with indentation and the first un-indented line marks the end of the loop.
- Python interprets any non-zero value as True. None and 0 are interpreted as False.
- At least one statement must be present in the loop body.
- '*else*' is executed when test condition evaluates to false and is optional.

Using *else* Statement with Loops

Python supports an *else* statement to be associated with a loop statement.

- If the *else* statement is used with a *for* loop, the *else* statement is executed when the loop has exhausted iterating the list.
- If the *else* statement is used with a *while* loop, the *else* statement is executed when the condition becomes false.

Now let us discuss what an Infinite loop in Python is.

Infinite Loop

A loop becomes an infinite loop if a condition never becomes false. You must use caution when using while loops because of the possibility that this condition never resolves to a false value. This results in a loop that never ends. Such a loop is called an infinite loop.

Practical Implementation-17

Illustrating the concept of Infinite Loop in *while* loop statement.

```
prgm11.py  /Users/ireeti/AppData/Local/Programs/Python/Python16-1/prgm11.py
File Edit Format Run Options Window Help
#Program to illustrate the concept of infinite loop using while loop construct
var = 1
while var == 1:      # Condition resulting in an infinite loop
    num = int(input("Enter a number: "))
    print("You entered : ", num)
    print("Good bye")
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.19
00 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
----- RESTART: C:\Chapter 5\prgm_11.py -----
Enter a number :100
You entered: 100
Enter a number :200
You entered: 200
Enter a number :500
You entered: 500
Enter a number :600
You entered: 600
Enter a number :
```

In the previous program, 'var' is a loop control variable and is assigned 1 before the loop starts executing. Since loop control variable is not updated in the body of loop, it will not exit a loop and becomes an infinite loop.

6.7 NESTED LOOPS

A loop may contain another loop as a part of its body. This loop inside a loop is called a nested loop. Once the condition of outer loop is true, the control is transferred to the inner loop. The inner loop is terminated first and then the outer loop terminates.

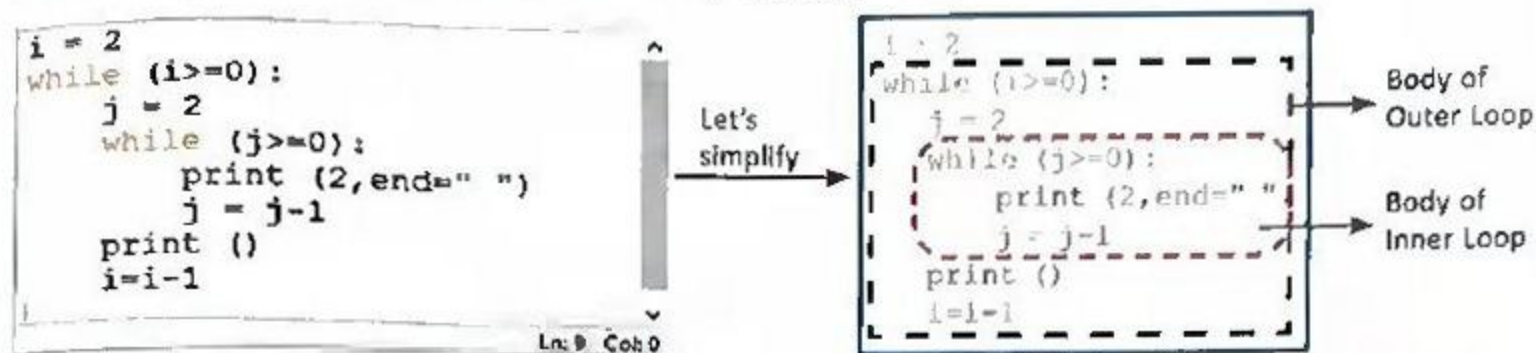
Syntax for a nested while loop:

```
while expression:
    while expression:
        statement(s)
    statement(s)
statement 2
```

In a nested *while* loop, a *while* loop is defined inside another *while* loop.

practical Implementation-18

Illustrating the concept of nested while loop statement.



*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.19
00 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/chapter 5/prgm_17.py
2 2 2
2 2 2
2 2 2
>>>
```

Explanation:

In the above program,

1. 'i' is a loop control variable for outer loop and is assigned a value 2 before the loop execution begins.
2. Test expression is checked for outer loop first.
3. The body of the outer loop is executed as test expression evaluates to true, it executes body of the outer while, which contains another while statement, i.e., inner while.
4. 'j' is a loop control variable for inner loop and is assigned 2 before inner loop execution. For i=0, inner loop will be executed 3 times and will print 2 2 2, after which the inner loop gets terminated.

After executing the rest of the body of outer loop, control goes to the next iteration of outer loop and 'i' is incremented and becomes 1, again executes the body of outer loop (first executes the inner loop). This process goes on till outer loop test expression remains True.

Syntax for a nested while loop:

```
for iterating_var in sequence:
    for iterating_var in sequence:
        statement(s)
    statement(s)
statement 2
```

Like while, we can place for loop inside another loop as shown in Practical Implementation-18.

Practical Implementation-19

Illustrating the concept of nested *for* loop statement.

```
prgm_18.py - C:/chapter 5/prgm_18.py (3.6.5)
File Edit Format Run Options Window Help
for i in range(1,4):
    for j in range(1,i+1):
        print (j,end=' ')
    print ()
Ln 2 Col 26
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Int
el)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/chapter 5/prgm_18.py
1
1 2
1 2 3
>>>
Ln 8 Col 4
```

Explanation:

Here, range function will generate value 1, 2, 3 in the outer loop. The inner loop will run for each value of 'i' used in outer loop. The value of outer loop variable will change only after the inner loop completely terminates.

POINT TO REMEMBER

- Nested loop is one loop inside another loop.
- Once the condition of outer loop is true, the control is transferred to the inner loop. First the inner loop is terminated and then the outer loop terminates.
- The inner loop must have a different name for its loop counter variable so that it will not conflict with the outer loop.

6.8 JUMP STATEMENTS

Looping allows a user to program and repeat tasks efficiently. In certain situations, when some particular condition occurs, a user may want to exit a loop (come out of the loop forever) or skip some statements of the loop before continuing further in the loop. These requirements can be taken care of by use of *break* and *continue* control statements respectively. Python provides these jump statements as a tool to give more flexibility to the programmer to control the logic of loops.

6.8.1 break Statement

The *break* statement terminates the current loop, i.e., the loop in which it appears, and resumes execution at the next statement immediately after the end of that loop. If *break* statement is inside a nested loop (loop inside another loop), *break* will terminate the innermost loop.

Syntax:

break

for **var** **in** **sequence**:

codes inside for loop

if **condition**:

break

codes inside for loop

codes outside for loop

while **test expression**:

codes inside while loop

if **condition**:

break

codes inside while loop

codes outside while loop

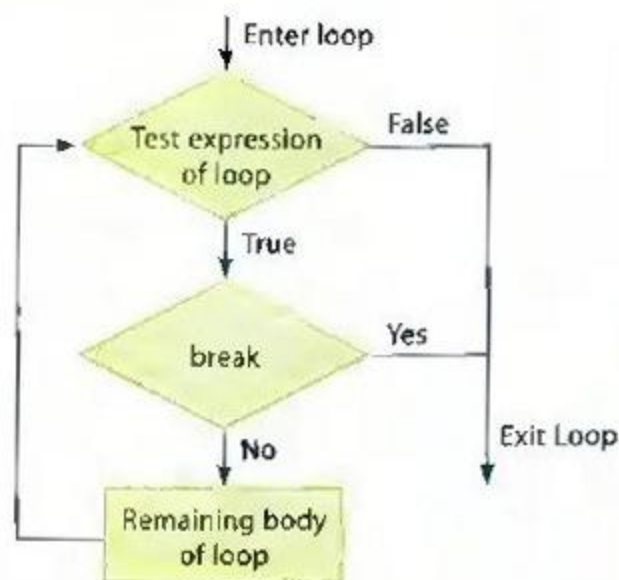


Fig. 6.12: Flow Chart for *break* Statement

Practical Implementation-20

Illustrating the working of *break* statement.

```
pgm_19.py - C:/Chapter 5/pgm_19.py (3.6.5)
File Edit Format Run Options Window Help
# Use of break statement inside loop
for val in "string":
    if val == "i":
        break
    print (val)
print ("Over")
Ln 6 Col 12
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
----- RESTART: C:/Chapter 5/pgm_19.py -----
s
t
r
Over
>>>
```

Explanation:

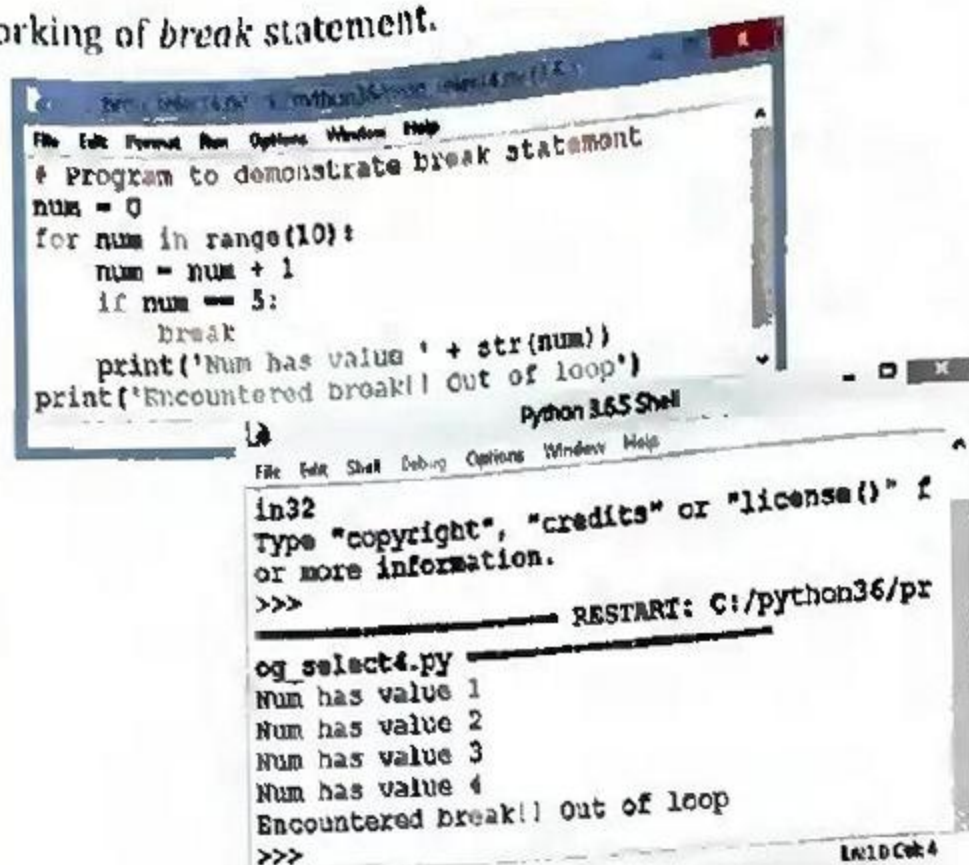
1. In this program, we iterate through the "string" sequence.
2. When letter is "i", it breaks the loop.
3. Hence, we see in our output that all the letters get printed till "i" after which the loop terminates.

It prints 'Over' the last statement of the program which is not a part of the *for* loop.



Practical Implementation-21

Illustrating the working of *break* statement.



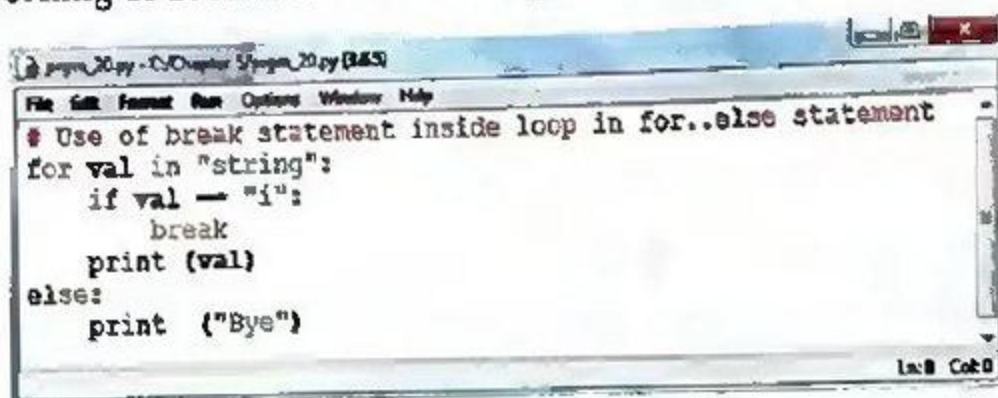
```
File Edit Format Run Options Window Help
# Program to demonstrate break statement
num = 0
for num in range(10):
    num = num + 1
    if num == 5:
        break
    print('Num has value ' + str(num))
print('Encountered break!! Out of loop')
```

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
In32
Type "copyright", "credits" or "license()" for
or more information.
>>>
----- RESTART: C:/python36/pr
og_select4.py -----
Num has value 1
Num has value 2
Num has value 3
Num has value 4
Encountered break!! Out of loop
>>>
```

It is clearly shown in the above program that when the variable *num* becomes 5, the *break* statement is executed and the *for* loop is terminated.

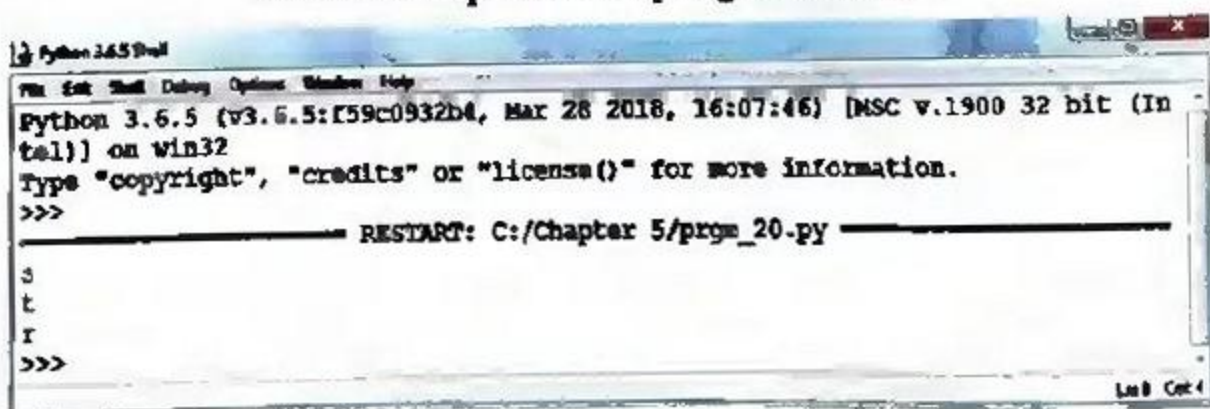
Practical Implementation-22

Illustrating the working of *break* statement with *for...else* statement.



```
File Edit Format Run Options Window Help
# Use of break statement inside loop in for..else statement
for val in "string":
    if val == "i":
        break
    print (val)
else:
    print ("Bye")
```

*****Output of the program*****



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (In
tel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
----- RESTART: C:/Chapter 5/prgm_20.py -----
s
t
r
>>>
```

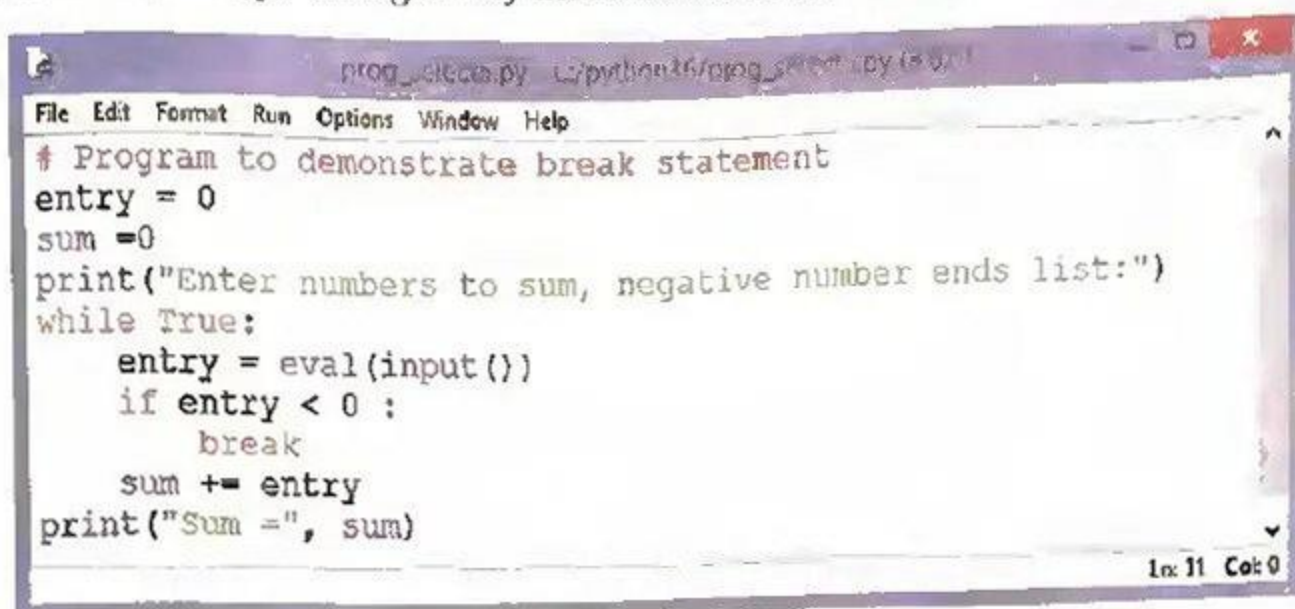
Explanation:

1. In this program, we iterate through the "string" sequence.
2. When letter is "i", it breaks the loop.
3. Hence, we see in our output that all the letters get printed before "i".

Here, *break* statement terminates the whole block of *for...else* and will not print "Bye".

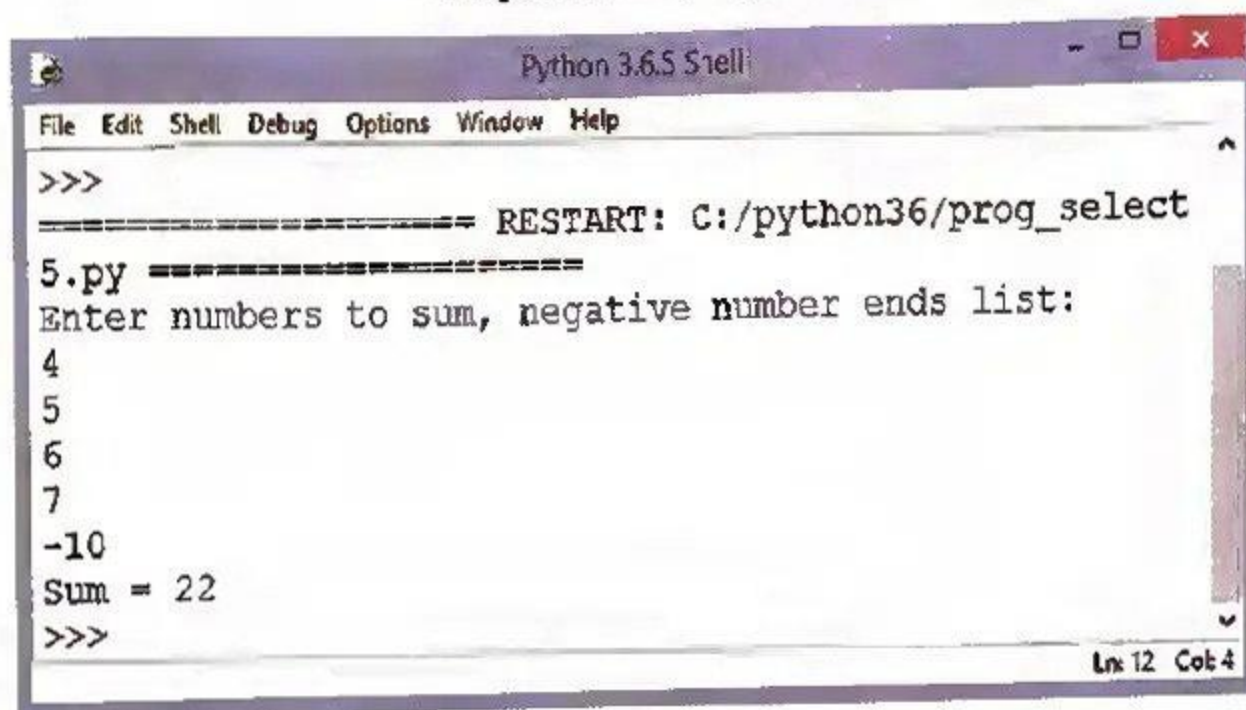
Practical Implementation-23

Find the sum of all the positive numbers entered by the user. As soon as the user enters a negative number, it stops taking in any more numbers from them and displays the sum.



```
prog_select.py C:/python36/prog_select.py (5.0)
File Edit Format Run Options Window Help
# Program to demonstrate break statement
entry = 0
sum = 0
print("Enter numbers to sum, negative number ends list:")
while True:
    entry = eval(input())
    if entry < 0 :
        break
    sum += entry
print("Sum =", sum)
```

*****Output of the program*****



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
>>>
===== RESTART: C:/python36/prog_select
5.py =====
Enter numbers to sum, negative number ends list:
4
5
6
7
-10
Sum = 22
>>>
```

In the above example, the condition of the *while* loop will always be true as it is set to value *true*, so, the body of the loop will always be entered and executed. Since the condition of the *while* can never be false, the *break* statement is the only way to get out of this loop. The *break* statement is executed when the user enters a negative number.

When a *break* statement is encountered during the execution of a program, the loop is instantly terminated. Any statement(s) following the *break* within the body of the loop are then skipped.

6.8.2 continue Statement

When a *continue* statement is encountered, the control jumps to the beginning of the loop for next iteration, thus skipping the execution of statements inside the body of loop for the current iteration. As usual, the loop condition is checked to see if the loop should continue further or terminate. If the condition of the loop is still true, the loop is entered again, else the control is transferred to the statement immediately following the loop.

Syntax:

`continue`

`for var in sequence:`

→ `# codes inside for loop`

`if condition:`

`continue`

`# codes inside for loop`

`# codes outside for loop`

`while test expression:`

→ `# codes inside while loop`

`if condition:`

`continue`

`# codes inside while loop`

`# codes outside while loop`

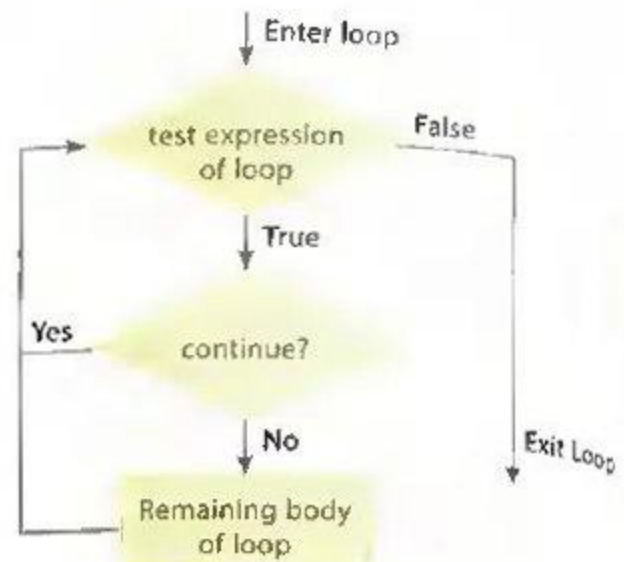


Fig. 6.13: Flow Chart for *continue* Statement

Practical Implementation-24

Illustrating the working of *continue* statement inside loops.

```
prgm_22.py - C:/Chapter 5/prgm_22.py (3.6.5)
File Edit Format Run Options Window Help
# Program to show the use of continue statement inside loops
for val in "string":
    if val == "i":
        continue
    print(val)
print("Over")
```

*****Output of the program*****

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (In
tel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
-----RESTART: C:/Chapter 5/prgm_22.py-----
s
t
r
i
n
g
Over
>>>
```

Explanation:

1. This program is same as the above example except that the *break* statement has been replaced with *continue*.
2. We continue with the loop if the string is "i", not executing the rest of the block.
3. Hence, we see in our output that all the letters are printed except "i".

After that, when the loop terminates, it prints 'Over' the last statement of the program which is not a part of *for* loop.

CTM: Like *break*, *continue* statement is within the block of code under the loop statement, usually preceded by an *if* statement.

6.8.3 pass Statement

It is used when a statement is required syntactically but you do not want any command or code to execute.

The **pass** statement is a *null* operation; nothing happens when it executes. The **pass** is also useful in places where your code will eventually go but has not been written yet. However, nothing happens when **pass** is executed. It results in no operation.

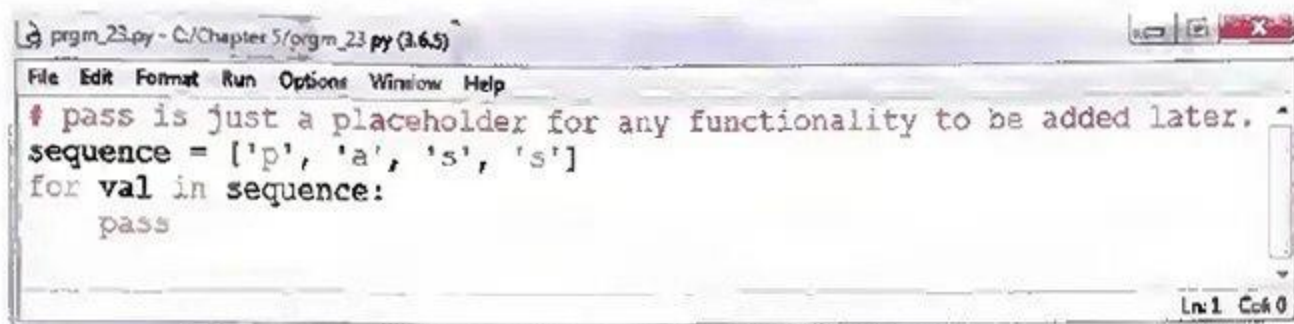
Syntax:

```
pass
```

Suppose we have a loop or a function that is not implemented yet but we want to implement it in the future. It cannot have an empty body. The interpreter would complain. So, we use the **pass** statement to construct a body that does nothing.

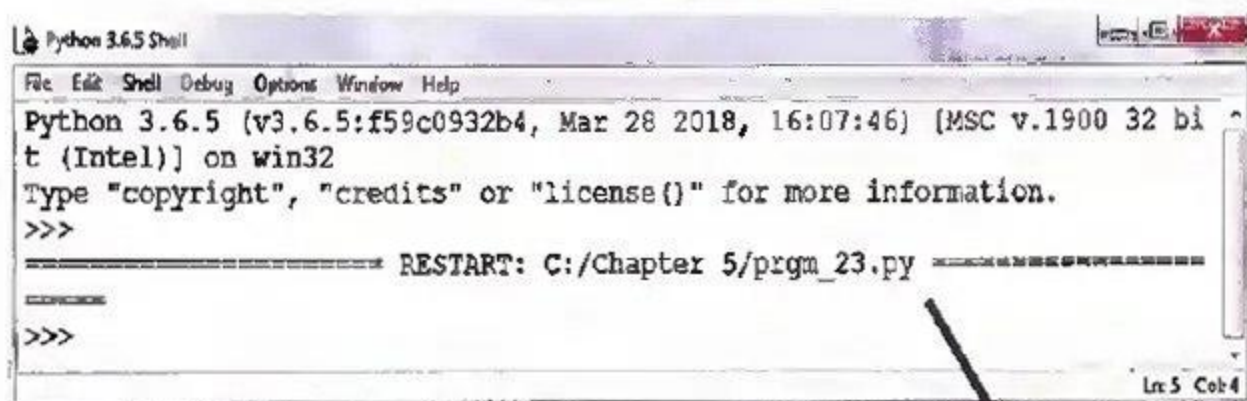
Practical Implementation-25

Illustrating the working of **pass** statement.



```
prgm_23.py - C:/Chapter 5/prgm_23.py (3.6.5)
File Edit Format Run Options Window Help
# pass is just a placeholder for any functionality to be added later.
sequence = ['p', 'a', 's', 's']
for val in sequence:
    pass
Ln:1 Col:0
```

*****Output of the program*****



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bi
t (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Chapter 5/prgm_23.py =====
>>>
```

No output

POINTS TO REMEMBER

- The *break* statement terminates the loop containing it.
- If *break* statement is inside a nested loop (loop inside another loop), *break* will terminate the inner-most loop.
- The *continue* statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.
- We use the **pass** statement to construct a body that does nothing.



MEMORY BYTES

- The three flow of control statements are sequential, selection and iteration
- The sequential statements get executed sequentially
- The selection statements depend upon conditions for execution
- Flow charts are diagrams that show the step-by-step solution to a given problem
- The *if* statement is a decision-making statement.
- The looping constructs *while* and *for* statements allow sections of code to be executed repeatedly
- *for* statement iterates over a range of values or a sequence.
- The statements within the body of a *while* loop are executed over and over until the condition of the *while* is false
- If the condition of the *while* loop is initially false, the body is not executed even once.
- The statements within the body of the *while* loop must ensure that the condition eventually becomes false, otherwise, the loop will become an infinite loop, leading to a logical error in the program.
- A loop contained within another loop is called a nested loop.
- The *break* statement immediately exits a loop, skipping the rest of the body of the loop. Execution continues with the statement immediately following the body of the loop.
- When a *continue* statement is encountered, the control jumps to the beginning of the loop for next iteration.
- The *pass* statement is a null operation; it does nothing.

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- (a) Statements in Python are executed while working with a sequence construct.
- (b) are diagrams that show the step-by-step solution to a given problem.
- (c) Two types of looping statements are and
- (d) loop is the best when the number of iterations is known.
- (e) statement terminates the loop.
- (f) A loop that never ends is called an loop.
- (g) The construct means repetition of a set of statements depending upon a condition test.
- (h) In nested loop loop must be terminated before the outer loop.
- (i) statement abandons the current iteration of the loop.
- (j) line begins with a keyword and ends with a colon.

2. State whether the following statements are True or False.

- (a) Flow chart is not a graphical representation of steps to solve a given problem.
- (b) *if*, *elif*, *else* are not block or compound statements.
- (c) Every compound statement of Python has a header and indented body below the header.
- (d) Block is the group of consecutive statements having same indentation level.
- (e) Python offers *pass* statement as empty statement.
- (f) To cancel the running of endless loop, you can press CTRL+C key any time during its endless repetition to stop it.
- (g) Indentation while working with blocks is not necessary in Python.
- (h) *else if* can be used in Python.
- (i) The *continue* statement skips the rest of the loop and causes the next iteration of loop to take place.
- (j) The *else* block for a loop executes only in the case of normal termination of the loop.
- (k) Repeated execution of a set of statements is called iteration.

3. Multiple Choice Questions (MCQs):

- (a) A graphical representation of an algorithm to solve a given problem:
(i) Flow chart (ii) Pie chart (iii) Bar chart (iv) Column chart
- (b) Which of the following is not a decision-making statement?
(i) if-elif statement (ii) for statement (iii) if-else statement (iv) if statement
- (c) Symbol used to end the if statement:
(i) Semicolon (;) (ii) Hyphen (-) (iii) Underscore (_) (iv) Colon (:)
- (d) Which of the following is a valid keyword?
(i) IF (ii) If (iii) if (iv) None of these
- (e) What does the following code print to console?

```
if True:
    print (101)
else:
    print (202)
```


(i) 101 (ii) 202 (iii) 303 (iv) 102
- (f) Which of the following is not a loop statement in Python?
(i) do-while (ii) while (iii) for (iv) All of these
- (g) How many times will the following code be executed?

```
a=5
while a > 0:
    print(a)
print("Thank You")
```


(i) 5 times (ii) Once (iii) Infinite (iv) None of these
- (h) Which statement is used to iterate itself over a range of values or a sequence?
(i) if (ii) while (iii) do-while (iv) for
- (i) Find the output of the following Python program:

```
for x in range(1, 20, 3):
    print(x)
```


(i) 1,20,3 (ii) 1,4,7,10,13,16,19 (iii) 13,6,9,12,15,18 (iv) 20,40,60,80,100
- (j) What abandons the current iteration of the loop?
(i) continue (ii) break (iii) stop (iv) infinite

SOLVED QUESTIONS

1. What is a statement?

Ans. A statement is a logical instruction which is executed while running a program.

2. How many kinds of statements are there?

Ans. Statements are categorized into three types:

- (a) Empty statements, (b) Simple statements, and
(c) Compound statements.

3. What is the significance of an empty statement?

Ans. An empty statement does nothing. In Python, an empty statement is a pass statement.

4. Define flow charts.

Ans. Flow charts are diagrams that show the step-by-step solution to a given problem. It is a pictorial representation of the solution to be developed for executing a task.

5. What is the difference between break and continue statements?

Ans. break and continue are the jump statements that can be used to alter the flow of loop.
break statement terminates the loop and resumes execution in the statement.
continue statement skips the rest of the code inside a loop for the current iteration.

6. What is the purpose of 'else' in a loop?

Ans. Looping statements can have an optional else block as well. The else part is executed if the items in the sequence do not match or when loop gets over.

7. Write logical expression for the following:

- (a) Either A is greater than B or A is less than C.
- (b) Name is Aman and age is between 18 and 35.
- (c) Place is either 'Delhi' or 'Goa' but not 'Jaipur'.

Ans. (a) if $A > B$ or $A < C$: (b) $Name == "Aman"$ and $(age \geq 18 \text{ and } age \leq 35)$
(c) $(Place == 'Delhi' \text{ or } Place == 'Goa')$ and $Place != 'Jaipur'$

8. Convert the following while loop into for loop:

```
x = 5
while (x < 10):
    print(x+10)
    x += 2
```

Ans. for x in range(5, 10, 2):
 print(x+10)

9. Convert the following for loop into while loop

```
for k in range(10, 20, 5):
    print(k)
```

Ans. $k = 10$
while $(k < 20)$:
 print(k)
 $k += 5$

10. Find error(s) in the following code (if any) and correct it by rewriting the code and underline the corrections.

```
(a) x = int("Enter value for x:")
for i in range(0, 11):
    if x = y:
        print(x*y)
    else:
        Print x-y
```

Ans. $x = \text{int}(\text{input}(\text{"Enter value for x: "}))$
for y in range(0, 11):
 if $x == y$:
 print(x*y)
 else:
 print(x-y)

```
(b) MaxSpeed=50
Alter=N
MySpeed=int(input())
If MySpeed >MaxSpeed
    Alter='Y':
Print(Alter)
```

Ans. $MaxSpeed = 50$
 $Alter = 'N'$
 $MySpeed = \text{int}(\text{input}())$
if $MySpeed > MaxSpeed$:
 $Alter = 'Y'$:
print(Alter)

```
(c) A=int(input("enter a value"))
while A!=0
    count=count+1
    if count<A
        print(count)
    else
        print(count+A)
    A=A-1
```

```

Ans. A=int(input("enter a value"))
while A!=0:
    count=count+1
    if count<A:
        print(count)
    else:
        print(count+A)
    A=A+1

```

11. Predict the output of the following code:

```

(a) x=3
    if x>2 or x<5 and x==6:
        print("ok")
    else:
        print("no output")

```

```

(b) x,y=2,4
    if (x+y==10)
        print("True")
    else:
        print("False")

```

```

(c) x = 10
    y = 0
    while x>y:
        x =x-4
        y += 4
        print(x, end=" ")

```

```

(d) i = 0
    while (i<10):
        i = i+1
        if i==5:
            break
        print(i, end=" ")

```

```

(e) for x in range(2):
    for y in range(2):
        print(x,y,x+y)

```

```

(f) x = 2
    y = 3
    for i in range(y*2-x):
        print(i,end=' ')

```

```

(g) for z in range(-100,200,100):
    print("*",z,end=' ')

```

```

(h) for z in range(-100,100,100):
    print(z,end=' ')

```

Ans. (a) ok	(b) False
(c) 6 2	(d) 1 2 3 4
(e) 0 0 0	(f) 0 1 2 3
0 1 1	
1 0 1	
1 1 2	
(g) * -100 * 0 * 100	(h) -100 0



12. Find the output of the following code:

```
x=0
z=1
for i in range(1,3):
    for j in range(1,i):
        z = i+j-1
        if(z%2==0):
            x=x+z
        elif(z%3==0):
            x=x+z-2
    print("x =",x)
```

Ans. x = 2

13. Find the output of the following code if user inputs the following values for year:

(a) 2000

(b) 1971

```
year = int(input('Enter 4-digit year: '))
if year%100==0:
    if (year%400==0):
        print('leap year')
    elif (year%4==0):
        print('leap year')
else:
    print("Not a leap year")
```

Ans. (a) Leap year

(b) Not a Leap year

14. Write the output of the following code:

```
x = 1
if x>3:
    if x>4:
        print('a')
    else:
        print('b')
elif x<2:
    if (x % 2) == 0:
        print('c')
    else:
        print('d')
```

Ans. c

d

15. Give the output of the following program

```
Data=["P",20,"R",10,"S",30]
Times=0
Alpha=""
Add=0
for C in range(1,len(Data)+1):
    Times=Times+C
    Alpha=Alpha+Data[C-1]+"@"
    Add=Add+Data[C-1]
print(Times, Add, Alpha)
```

Ans. 60 P@R@S@

16. How many times will the following loop be executed.

```
for I in range(1,51,3):
    print(I)
```

Ans. 16 times

17. WAP to convert temperature from Celsius to Fahrenheit. Also write pseudo-code for the same.

Ans. `Celsius = float(input("Enter temperature in Celsius:"))
Fahrenheit = (Celsius * 1.8) + 32 # calculate Fahrenheit
print(Celsius, " Celsius", Fahrenheit, "Fahrenheit")`

Pseudo-code:

Input Celsius

Set Fahrenheit by multiplying Celsius by 1.8 and then adding 32

Print the Celsius and Fahrenheit.

18. WAP to calculate and display Area and Perimeter of a rectangle.

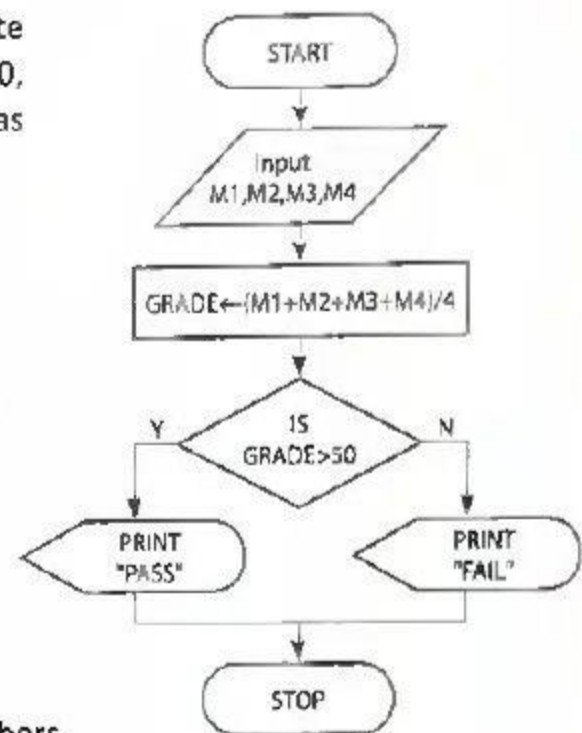
Ans. Code:

```
# Python Program to find Area and Perimeter of a Rectangle
length = float(input('Please Enter the length of a Rectangle:'))
breadth = float(input('Please Enter the breadth of a Rectangle:'))
Area = length * breadth # calculate the area
Perimeter = 2 * (length + breadth) # calculate the Perimeter
print("Area of a Rectangle is: ", Area)
print("Perimeter of Rectangle is: ", Perimeter)
```

19. Draw a flow chart to determine a student's final grade and indicate whether they are passing or failing. If the grade is greater than 50, then print "Pass" else print "Fail". The final grade is calculated as the average of marks of four subjects.

Ans. Code:

```
m1 = int(input("Enter marks of subject 1 : "))
m2 = int(input("Enter marks of subject 2 : "))
m3 = int(input("Enter marks of subject 3 : "))
m4 = int(input("Enter marks of subject 4 : "))
grade = (m1 + m2 + m3 + m4) / 4
if grade > 50:
    print("Pass")
else:
    print("Fail")
```



20. WAP to find the largest number among the three inputted numbers.

Ans. Code:

```
# Python program to find the largest number among the three inputted numbers
A = int(input("Enter first number: "))
B = int(input("Enter second number: "))
C = int(input("Enter third number: "))
if (A > B):
    if (A > C):
        print(A, " is the largest number")
    else:
        print(C, " is the largest number")
else:
    if (B > C):
        print(B, " is the largest number")
    else:
        print(C, " is the largest number")
```

21. WAP to display sum of even numbers up to number n entered by the user.

Ans. `s=0`

```
n=int(input('Enter a limit'))
for i in range(n+1):
    if i%2==0:
        s+=i
print(s)
```

22. WAP that reads two numbers and an arithmetic operator and displays the results.

Ans. Code:

```
a = float(input('Enter first number:'))
b = float(input('Enter second number:'))
Op = input('Enter the operator:')
if Op=='+':
    print(a+b)
elif Op=='-':
    print(a-b)
elif Op=='*':
    print(a*b)
elif Op=='/':
    print(a/b)
elif Op=='%':
    print(a%b)
elif Op=='**':
    print(a**b)
else:
    print('Invalid option')
```

23. WAP to perform the following task according to the user's choice using menu.

- | | |
|-----------------------------|-----------------------|
| (a) Area of Circle | (b) Area of Rectangle |
| (c) Circumference of Circle | (d) Area of Square |

Ans. while True:

```
print(" 1. For Area of Circle")
print(" 2. For Area of Rectangle")
print(" 3. For Circumference of Circle")
print(" 4. For Area of Square")
ch=int(input("Enter your choice (enter 0 to exit): "))
if ch==1:
    r=int(input("Enter the radius of the Circle:"))
    a=3.14*r*r
    print("Area of Circle: ",a)
elif ch==2:
    l=int(input("Enter the length:"))
    b=int(input("Enter the breadth:"))
    a=l*b
    print("Area of Rectangle: ",a)
elif ch==3:
    r=int(input("Enter the radius of the Circle:"))
    c=2*(int(3.14*r))
    print("Circumference of Circle: ",c)
```

```

elif ch==4:
    l=int(input("Enter the side of square:"))
    a=l*l
    print("Area of Square: ",a)
elif ch==0:
    break
else:
    print("Invalid Option")

```

24. Write a program to accept a character from the user and display whether it is a vowel or consonant.

Ans. `ch = input("Enter a character:")`

```

if (ch == 'a' or ch == 'A'):
    print(ch, "is a vowel")
elif (ch == 'e' or ch == 'E'):
    print(ch, "is a vowel")
elif (ch == 'i' or ch == 'I'):
    print(ch, "is a vowel")
elif (ch == 'o' or ch == 'O'):
    print(ch, "is a vowel")
elif (ch == 'u' or ch == 'U'):
    print(ch, "is a vowel")
else:
    print(ch, "is a consonant")

```

25. WAP to accept a number from the user and print the table of that number.

Ans. Code:

```

x = int(input("Enter no for which table is required:"))
i = 1
while (i<=10) :
    print(x, " x ", i, " = ", x*i)
    i+=1

```

26. WAP to print Fibonacci series up to a certain limit.

Ans. Code:

```

i = int(input("Enter the limit:"))
x = 0
y = 1
z = 1
print("Fibonacci series\n")
print(x,y,end= " ")
while(z<=i):
    print(z, end=' ')
    x = y
    y = z
    z=x+y

```

27. Write a program to enter a number and check if it is a prime number or not.

Ans. Code:

```

num=int(input('Enter the number:'))
for i in range(2,num//2+1):
    if num%i==0:
        print('It is not a prime number')
        break
else:
    print('It is a prime number')

```

28. WAP to accept a number, find out and display whether it is an Armstrong number or not
Note: An Armstrong number of three digits is an integer such that the sum of the cubes of its digits is equal to the number itself. For example, 371 is an Armstrong number since $3^3 + 7^3 + 1^3 = 371$.

Ans. Code:

```
num=int(input("Enter 3-digit number: "))
f = num
sum = 0
while(f>0):
    a = f%10
    f = int(f/10)
    sum = sum+(a**3)
if (sum==num):
    print(num, " is an Armstrong number.")
else:
    print(num, " is not an Armstrong number.")
```

29. Write a program to accept a number and display whether the number is a Palindrome or not.

Ans. Code:

```
orig = int(input("Enter a number: "))
num = orig
rev = 0
while num > 0:
    digit = num % 10
    rev = rev * 10 + digit
    num = int(num/10)
if orig == rev:
    print('Palindrome')
else:
    print('Not a Palindrome')
```

30. WAP to accept a number and find out whether it is a perfect number or not.

Note: Perfect number is a positive integer that is equal to the sum of its proper divisors. The smallest perfect number is 6.

Ans. Code:

```
i = 1
s = 0
num = int(input("Enter number:"))
while i<num:
    if num%i == 0:
        s += i
    i = i+1
if s == num:
    print("It is a perfect no.")
else:
    print("It is not a perfect no.")
```

31. WAP to accept a decimal number and display its binary number.

Ans. Code:

```
n = int(input("Enter no:"))
a = 0
m = 0
c = 1
while n > 0:
    a = n%2
    m = m + (a*c)
    c = c*10
    n = int(n/2)
print(m)
```

32. WAP to print the sum of the series $1+x^1/1!+x^2/2!+.....x^n/(n)!$ —exponential series.

Ans. Code:

```
x = int(input("Enter the value of x :"))
n = int(input("Enter the limit :"))
total = 1
for i in range(1, n + 1):
    total = total + ((x**i)/i)
print(round(total,2))
```

33. WAP to find the sum of the geometric series given below:

$$S = a+ar+ar^2+ar^3+ar^4+.....ar^n$$

Ans. Code:

```
n = int(input("Enter max value of the series:"))
r = float(input("Enter ratio:"))
a = int(input("Enter the first term:"))
print ("\n Series:\n")
sum=0
for i in range(1,n):
    term=a*(r**i)
    sum=sum+term
print("Sum of the series: ", sum)
```

34. WAP to print the following pattern:

```
5 5 5 5
5 5 5 5
5 5 5 5
5 5 5 5
```

Ans. i = 3

```
while (i>=0):
    j = 3
    while (j>=0):
        print(5,end=' ')
        j=j-1
    print()
    i=i-1
```

35. WAP to print the following pattern:

```
1 2 3
1 2
1
```

Ans. i = 3

```
while i>=0:
    j = 1
    while j<=i:
        print(j,end=' ')
        j= j+1
    print()
    i=i-1
```

36. WAP to print the following pattern:

```
A
B C
D E F
G H I J
K L M N O
```

Ans. i=1

```
num=65
```

```
while i<=5:
```

```
    j = 1
```

```
    while j<=i:
```

```
        print(chr(num),end=" ")
```

```
        j = j+1
```

```
        num=num+1
```

```
    print()
```

```
    i=i+1
```

37. WAP to print the following patterns:

(i) 1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

(ii) 1

3 3

5 5 5

7 7 7 7

9 9 9 9 9

(iii) 5

4 5

3 4 5

2 3 4 5

1 2 3 4 5

Ans. (i) n=5

```
for i in range(1,n+1):
```

```
    for j in range(1,i+1):
```

```
        print(j,end='')
```

```
    print("")
```

(ii) n=10

```
for i in range(1,n+1,2):
```

```
    for j in range(0,i+1,2):
```

```
        print(i,end='')
```

```
    print("")
```

(iii) space=4

```
for a in range(1,5):
```

```
    print(" "*space,end="")
```

```
    for x in range(a,6):
```

```
        print(a,end="")
```

```
    space-=1
```

38. Convert the following *while* loop to *for* loop:

```
a=3
```

```
while a<100:
```

```
    if a%2==0:
```

```
        print(a+2,end=",")
```

```
    else:
```

```
        print(a*2,sep="$")
```

```
    a=a+3
```

```
Ans. for a in range(3,100,3):
    if a%2==0:
        print(a+2,end=" ")
    else:
        print(a*2,sep="$")
```

39. Complete the following code to reverse a number:

```
n=int(input("Enter the number:"))
rev=0
while _____:
    x=_____
    rev=_____
    n=_____
print("Reverse number=",rev)
```

Ans. (i) $n!=0$
 (ii) $x=x\%10$
 (iii) $rev=rev*10+x$
 (iv) $n=n//10$

40. Make the correction in the following loop so that it prints multiples of 5:

```
for a in range(5,100):
    print(a)
```

Ans. for a in range(5,100,5):
 print(a)

41. Write a program to accept 10 numbers and find the maximum value entered by the user. It should display the maximum number after accepting 10 values.

```
Ans. n=int(input("Enter first number:"))
max=n
for i in range(2,10):
    n=int(input("Enter next number:"))
    if max<n:
        max=n
print("Maximum=",max)
```

42. Write a program to accept a character and display if it is an uppercase character, lowercase character, digit or a special character.

Input	Output
A-Z	uppercase character
a-z	lowercase character
0-9	digit
Any other character	special character

```
Ans. ch=input("Enter character:")
if ch.isupper():
    print("Uppercase character")
elif ch.islower():
    print("lowercase character")
elif ch.isdigit():
    print("Digit")
else:
    print("Special Character")
```

43. Write a program in Python to accept name and password from the user and display "You are admin" if name="admin" and password="root", and display "You are user" if name="user" and password="root". Display "You are Guest" if name is "guest", otherwise display "Not a valid user".

Ans.

```
name=input("enter username:")
password=input("enter password")
if name=='admin' and password=='root':
    print("you are admin")
elif name=="user" and password=="root":
    print("you are user")
elif name=="guest":
    print("you are guest")
else:
    print("not a valid user")
```

UNSOLVED QUESTIONS

1. What are compound statements?
2. What are jump statements?
3. What is nested loop? Explain with example.
4. What is the significance of updating loop control variable in while statements?
5. Construct a logical expression to represent each of the following conditions.
 - (i) Mark is greater than or equal to 100 but less than 70.
 - (ii) Num is between 0 and 5 but not equal to 2.
 - (iii) Answer is either 'N' OR 'n'.
 - (iv) Age is greater than or equal to 18 and gender is male.
 - (v) City is either 'Kolkata' or 'Mumbai'.
6. Find error in the following code (if any) and correct it by rewriting the code and underline the corrections:

```
code=input("Enter season code:")
if code=w:
    print "winter season"
    elif code==r:
        PRINT "rainy season"
else:
    Print "summer season"
```

7. Write the output of the following:

- (i)

```
for i in '123':
    print("guru99",i,)
```
- (ii)

```
for i in [100,200,300]:
    print(i)
```
- (iii)

```
for j in range(10,6,-2):
    print(j*2)
```
- (iv)

```
for x in range(1,6):
    for y in range(1,x+1):
        print(x, ' ',y)
```
- (v)

```
for x in range(10,20):
    if (x == 15):
        break
    print(x)
```

```
(vi) for x in range(1, 10):
    if x % 2 == 0:
        print(x)
    else:
        print(x)
```

8. Write the output of the following program for $x = 50$

```
if x < 10:
    if x < 5:
        print('x')
    if x < 40:
        print('y')
    elif x < 40:
        print('average')
    else:
        print('no output')
```

9. Write the output of the following code

```
if 1 in range(2):
    for j in range(1):
        if 1+2 == j:
            print("+", end=" ")
        else:
            print("o", end=" ")
```

10. Give the output of the following when $\text{num1} = 4$, $\text{num2} = 3$, $\text{num3} = 2$

- $\text{num1} += \text{num2} + \text{num3}$
- $\text{print}(\text{num1})$
- $\text{num1} = \text{num1} ** (\text{num2} + \text{num3})$
- $\text{print}(\text{num1})$
- $\text{num1} **= \text{num2} + \text{c}$
- $\text{num1} = '5' + '5'$
- $\text{print}(\text{num1})$
- $\text{print}(4.00 / (2.0 + 2.0))$
- $\text{num1} = 2 + 9 * ((3 * 12) - 8) / 10$
- $\text{print}(\text{num1})$
- $\text{num1} = \text{float}(10)$
- $\text{print}(\text{num1})$
- $\text{num1} = \text{int}('3.14')$
- $\text{print}(\text{num1})$
- $\text{print}(10 != 9 \text{ and } 20 >= 20)$
- $\text{print}(5 \% 10 + 10 < 50 \text{ and } 29 <= 29)$

11. What is the difference between *else* and *elif* construct of *if* statement?

12. Find the output of the following program segments:

- for i in range(20, 30, 2):
 print(i)
- country = 'INDIA'
for i in country:
 print(i)

```

(c) i = 0; sum = 0
    while i < 9:
        if i % 4 == 0:
            sum = sum + i
        i = i + 2
    print (sum)

```

13. Write a program to check if the number is positive or negative and display an appropriate message.
14. WAP to convert temperature in Fahrenheit to Celsius.
15. WAP to display even numbers between 10 and 20.
16. WAP to perform all the mathematical operations of a calculator.
17. Write a program to accept a number and display the factorial of that number.
18. WAP to convert binary number to decimal number.
19. WAP to find the sum of the digits of a number.
20. Write a function to display prime numbers below 30.
21. WAP to print the sum of the series $1 - x^1/2! + x^2/3! - x^3/4! + \dots + x^n/(n+1)!$ —exponential series.
22. WAP to print the sum of the series $1 - x^2/4! + x^3/6! - x^4/8! + x^5/10! + \dots + x^n/(2n)!$ —exponential series.
23. WAP to display the sum of the given series:
Sum = $1 + (1 + 2) + (1 + 2 + 3) + (1 + 2 + 3 + \dots + n)$

24. WAP to print the following pattern:

```

*
* *
* * *
* * * *
* * * * *

```

25. WAP to print the following pattern:

```

A
B B
C C C
D D D D
E E E E E

```

26. WAP to print the following pattern:

```

1
2 1
3 2 1
4 3 2 1
5 4 3 2 1

```

CASE-BASED/SOURCE-BASED INTEGRATED QUESTIONS

1. Guruji Astro is an astrological firm that provides complete guidance on Indian Astrology, and Free Horoscope Predictions.

In order to provide computerized horoscopes within no time, the firm hired a software developer to develop a Python program for the above requirement. Help him to write a program to display astrological sign for given date of birth.

Ans.

```

prog_astrological_horoscope.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37-32_
File Edit Format Run Options Window Help
#Program to display astrological sign for given date of birth
day = int(input("Enter Birthday: "))
month = input("Input month of birth (e.g. march, july etc): ")
if month == 'january':
    astro_sign = 'Capricorn' if (day < 22) else 'aquarius'
elif month == 'february':
    astro_sign = 'Capricorn' if (day < 20) else 'aquarius'
elif month == 'march':
    astro_sign = 'Aries' if (day < 21) else 'taurus'
elif month == 'april':
    astro_sign = 'Aries' if (day < 20) else 'taurus'
elif month == 'may':
    astro_sign = 'Taurus' if (day < 21) else 'gemini'
elif month == 'june':
    astro_sign = 'Gemini' if (day < 21) else 'cancer'
elif month == 'july':
    astro_sign = 'Cancer' if (day < 23) else 'leo'
elif month == 'august':
    astro_sign = 'Leo' if (day < 23) else 'virgo'
elif month == 'september':
    astro_sign = 'Virgo' if (day < 23) else 'libra'
elif month == 'october':
    astro_sign = 'Libra' if (day < 23) else 'scorpio'
elif month == 'november':
    astro_sign = 'scorpio' if (day < 22) else 'sagittarius'
print("Your Astrological sign is :",astro_sign)
Ln:31 Col:0

```

```

Python 3.7.0 Shell
File Edit Shell Debug Options Window Help
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Py
thon/Python37-32/prog_astrological_horoscope.py
Input birthday: 12
Input month of birth (e.g. march, july etc): april
Your Astrological sign is : Aries
>>>
Ln:8 Col:4

```

2. Redwood Shop deals in apparels and footwears. Write a program to calculate total selling price after levying of GST. To calculate the net price, Central Government and State Government GST rates as applicable are given as under:

Item	GST Rate
Footwear <= 500 (per pair)	5%
Footwear >= 500 (per pair)	18%
Apparels <= 1000 (per piece)	5%
Apparels >= 1000 (per piece)	12%

Computer Science with Python-XI
0.46

```
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python37-32.
ulation.py
Enter item code (A) Apparel, (F) Footwear :A
Enter selling price (per unit) of item:800

-----
                        INVOICE
-----

Item:                                Apparel
Price:                               800.00
CGST(8, (gstRate/2), %):             20.00
SGST(8, (gstRate/2), %):             20.00
-----
Amount payable:                       840.00
-----
>>>
```